

CCA103 SAD DOMAIN MODELING

Chapter 4

CHAPTER 4: OUTLINE

Part I: “Things” in the Problem Domain

Part II: The Entity-Relationship Diagram

Part III: The Domain Model Class Diagram

Part IV: The State Machine Diagram

LEARNING OBJECTIVES

1. Explain how the concept of “things” in the problem domain also defines requirements
2. Identify and analyze **data entities** and **domain classes** needed in the system
3. Read, interpret, and create an **entity-relationship** diagram
4. Read, interpret, and create a **domain model class** diagram
5. Understand the domain model class diagram
6. Read, interpret, and create a **state machine diagram** that models object behavior

THINGS IN THE PROBLEM DOMAIN

1. Problem domain is the specific area (or domain) of the users' business need that is within the scope of the new system.
2. “Things” are those items users work with when accomplishing tasks that need to be remembered
3. Examples of “Things” are **products**, **sales**, **shippers**, **customers**, **invoices**, **payments** or others.
4. These “Things” are modeled as **domain classes** or **data entities**
5. In this course, we will call them domain classes.
6. In the database class, you may call them data entities

THINGS IN THE PROBLEM DOMAIN:TWO TECHNIQUES FOR IDENTIFYING THEM

Brainstorming Technique

Use a checklist of all the usual types of things typically found and brainstorm to identify domain classes of each type

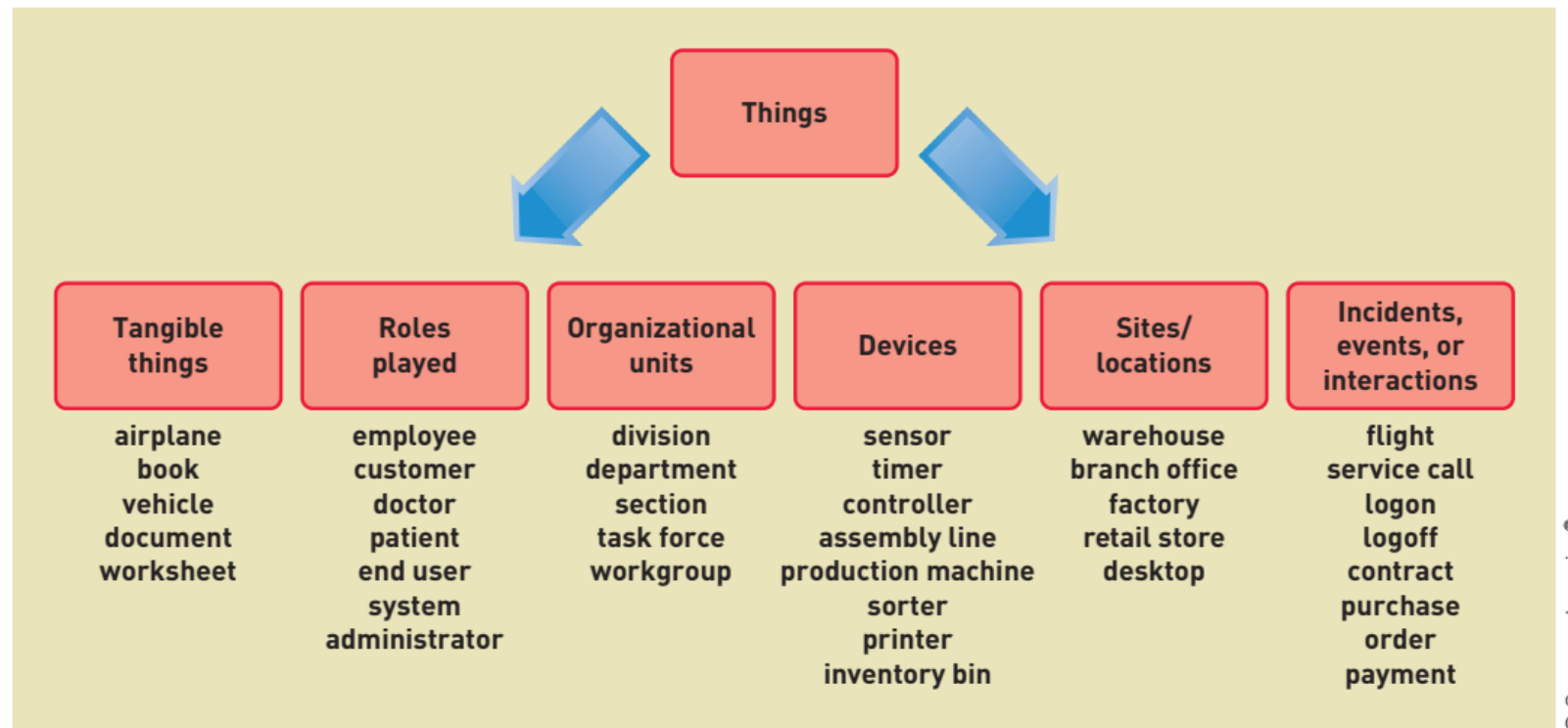
Noun Technique

Identify all the nouns that come up when the system is described and determine if each is a domain class, an attribute, or not something we need to remember

BRAINSTORMING TECHNIQUE

- Are there any tangible things?
- Are there any organizational units?
- Sites/locations?
- Are there incidents or events that need to be recorded?

FIGURE 4-1 *Types of things to use for the brainstorming technique*



BRAINSTORMING TECHNIQUE: STEPS

1

Identify a **user** and set of **use cases**



2

Brainstorm with the **user**
to identify things involved in their **use case**



3

Use **types of things (categories)** to ask
specific questions:

- Tangible items to store info?
- Are any locations involved?
- Roles played by people?



4

Continue to work with all users &
stakeholders to expand brainstorming list



5

Merge the results, eliminate duplicates,
compile an initial list



6

Continue to work with all users &
stakeholders to expand brainstorming list

THE NOUN TECHNIQUE

1. A technique to identify problem domain classes (things) by **finding**, **classifying**, and **refining** a list of **nouns** that come up in discussions or documents
2. Popular technique → Systematic.
3. Does end up with long lists and many nouns that are not things that need to be stored by the system
4. Difficulty identifying synonyms and things that are really attributes
5. Good places to start when there are no users available to help brainstorm

THE NOUN TECHNIQUE: STEPS #1

Identify all nouns in the system using use cases, actors, and other information.



1

Using the use cases, actors, and other information about the system—including inputs and outputs—identify all nouns.



Examples of nouns you might find:



customer



product item



sale



confirmation



transaction



shipping



bank



report



2

Using other information from existing systems, current procedures, current reports or forms, add items or categories of information needed.



Examples of additional details to include:



price



size



color



style



season



inventory
quantity



payment
method



shipping
address



Collect use cases



Identify actors



Review inputs & outputs



List all nouns



Validate & refine

THE **NOUN** TECHNIQUE: STEPS #2

Questions to refine it



- ✓ Is it a unique thing the system needs to know about?
- ✓ Is it inside the scope of the system I am working on?
- ✓ Does the system need to remember more than one of these items?

Questions to exclude it



- ✓ Is it really a synonym for some other thing I have identified?
- ✓ Is it just an output of the system produced from other information I have identified?
- ✓ Is it just an input that results in recording some other information I have identified?

Questions to research it



- ✓ Is it likely to be a specific piece of information (attribute) about some other thing I have identified?
- ✓ Is it something I might need if assumptions change?

--- Refine and validate your noun list with strategic questions. ---

Case of Course Registration System

Using the Noun Technique



What to Include?

Nouns to store as key data:

- ✓ Student
- ✓ Course
- ✓ Lecturer
- ✓ Section
- ✓ Registration
- ✓ Semester

Why Include?

- ✓ Essential for the system
- ✓ Found in multiple records



What to Exclude?

Nouns that are **NOT** main data objects:

- ✗ Registration form
- ✗ Confirmation slip
- ✗ Search result
- ✗ Course list
- ✗ Input field

Why Exclude?

- ✗ Duplicate or output data
- ✗ Not core to the system



What to Research?

Nouns to investigate further:

- ? Prerequisite
- ? Credit Limit
- ? Fee Status
- ? Registration Status
- ? Academic Standing

Why Research?

- ✓ Possible rules or attributes
- ✓ May affect other entities



Organize and validate your noun list effectively.

PARTIAL LIST OF NOUNS

Identified noun	Notes on including noun as a thing to store
Accounting	We know who they are. No need to store it.
Back order	A special type of order? Or a value of order status? Research.
Back-order information	An output that can be produced from other information.
Bank	Only one of them. No need to store.
Catalog	Yes, need to recall them, for different seasons and years. Include.
Catalog activity reports	An output that can be produced from other information. Not stored.
Catalog details	Same as catalog? Or the same as product items in the catalog? Research.
Change request	An input resulting in remembering changes to an order.
Charge adjustment	An input resulting in a transaction.
Color	One piece of information about a product item.
Confirmation	An output produced from other information. Not stored.
Credit card information	Part of an order? Or part of customer information? Research.
Customer	Yes, a key thing with lots of details required. Include.
Customer account	Possibly required if an RMO payment plan is included. Research.
Fulfillment reports	An output produced from information about shipments. Not stored.
Inventory quantity	One piece of information about a product item. Research.
Management	We know who they are. No need to store.
Marketing	We know who they are. No need to store.
Merchandising	We know who they are. No need to store.

© Cengage Learning®

DETAILS ABOUT DOMAIN CLASSES

1. Attribute normally describes one piece of information about each instance of the class
 - ✓ Customer has first name, last name, phone number
2. Identifier or key
 - ✓ One attribute uniquely identifies an instance of the class.
 - ✓ Required for data entities, optional for domain classes. **Customer ID** identifies a **customer**
3. Compound attribute (<1 attribute)
 - ✓ Two or more attributes combined into one structure to simplify the model.
 - ✓ For example, address rather than including number, street, city, state, zip separately.
 - ✓ Sometimes an identifier or key is a compound attribute.

ATTRIBUTES AND VALUES

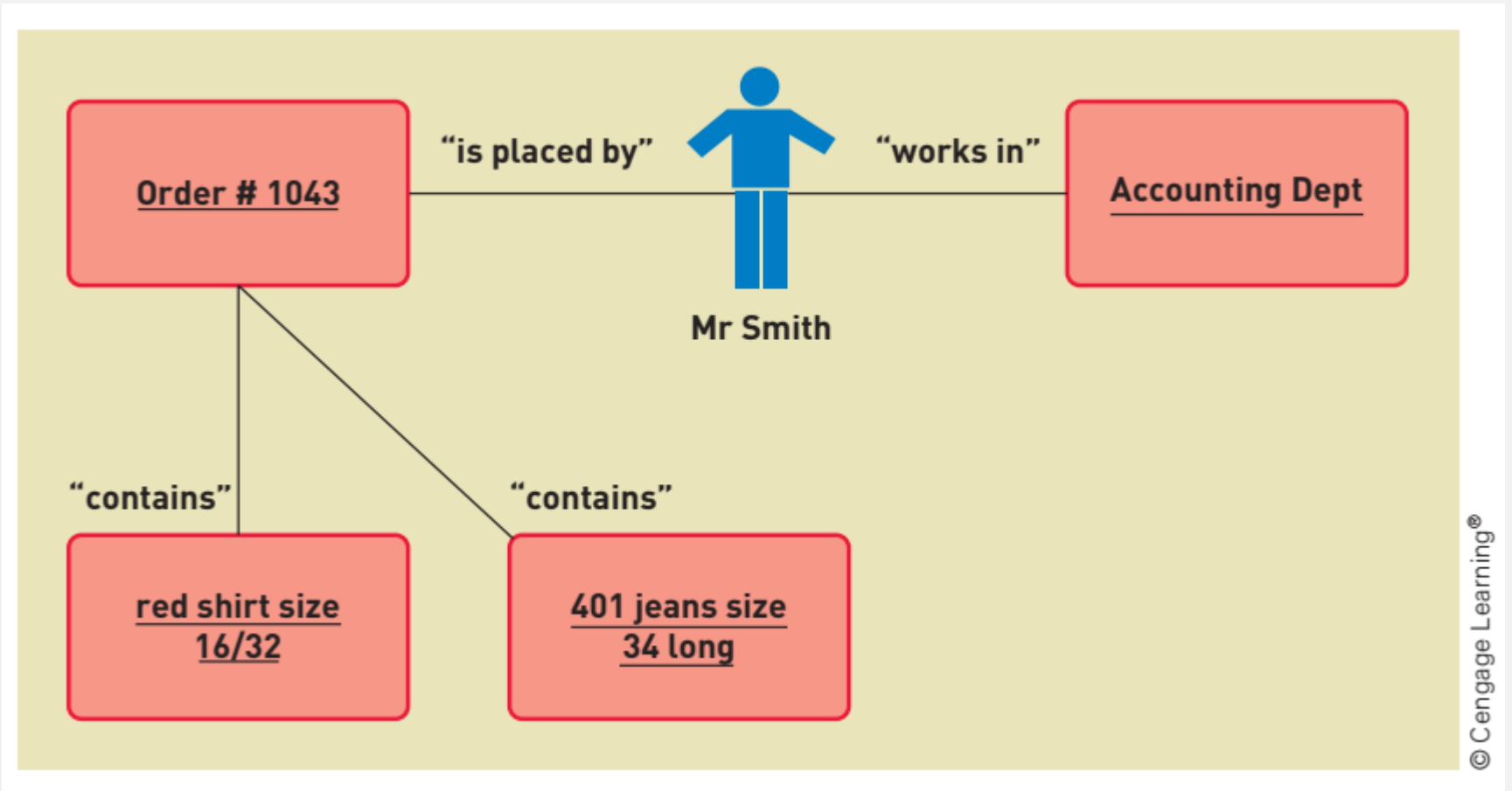
1. Class is a type of thing.
2. Object is a specific instance of the class.
3. Each instance has its own values for an attribute

All customers have these attributes:	Each customer has a value for each attribute:		
Customer ID	101	102	103
First name	John	Mary	Bill
Last name	Smith	Jones	Casper
Home phone	555-9182	423-1298	874-1297
Work phone	555-3425	423-3419	874-8546

© Cengage Learning®

ASSOCIATIONS AMONG THINGS

Association is a naturally occurring relationship between classes (UML term)



SOME CLARIFICATIONS....

Association x Class Diagram

Called the **association** on the **class** diagram in UML



UML

- **Multiplicity** is a term for the number of associations between classes: 1 to 1 or 1 to many
- UML is the primary emphasis of this text

1..1
1..* many

Relationship x ERD

Called the **relationship** on the **ERD** in the **database** class



ERD

- **Cardinality** is a term for the number of relationships in entity relationship diagrams: 1 to 1 or 1 to many

1..1
0..* many

Associations and Relationships

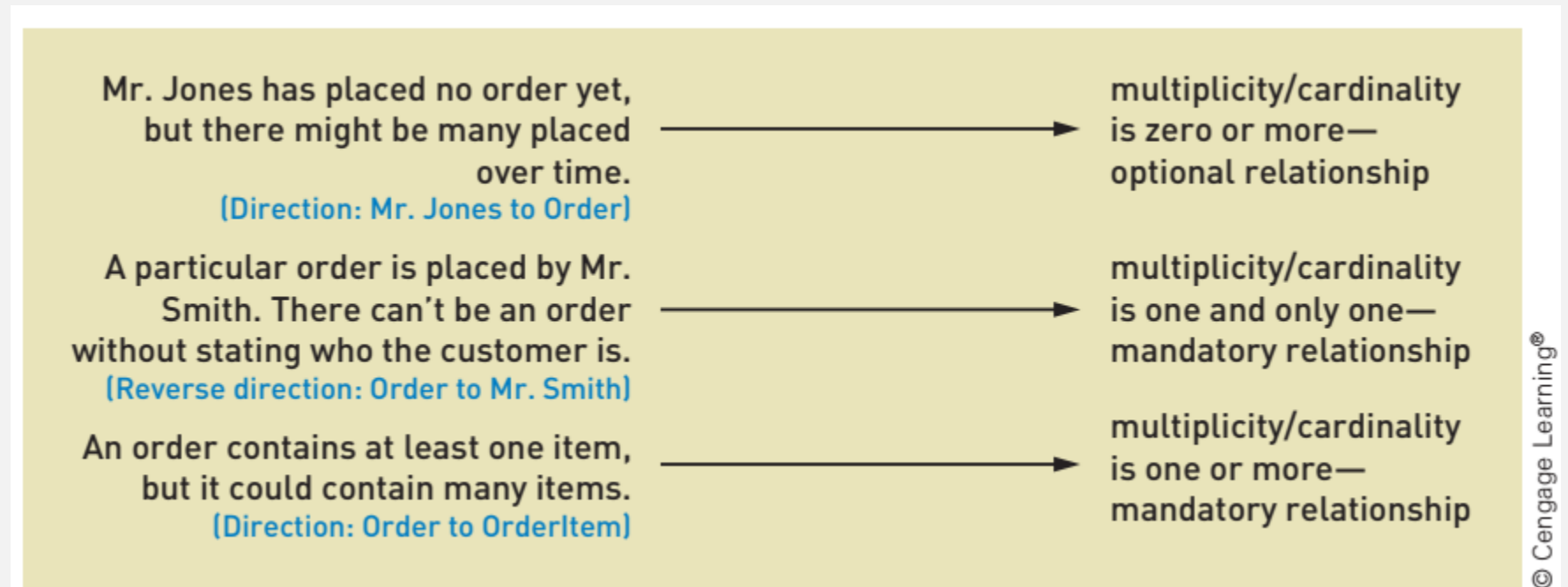
Associations and Relationships apply in two directions

Read them separately *each*:

- A customer places an order
- An order is placed by a customer

ASSOCIATION - MINIMUM AND MAXIMUM MULTIPLICITY

- Associations have minimum and maximum constraints,
 - **minimum** is **zero**, the association is **optional**
 - If the **minimum** is at least **one**, the association is **mandatory**



ASSOCIATION - TYPES OF ASSOCIATIONS

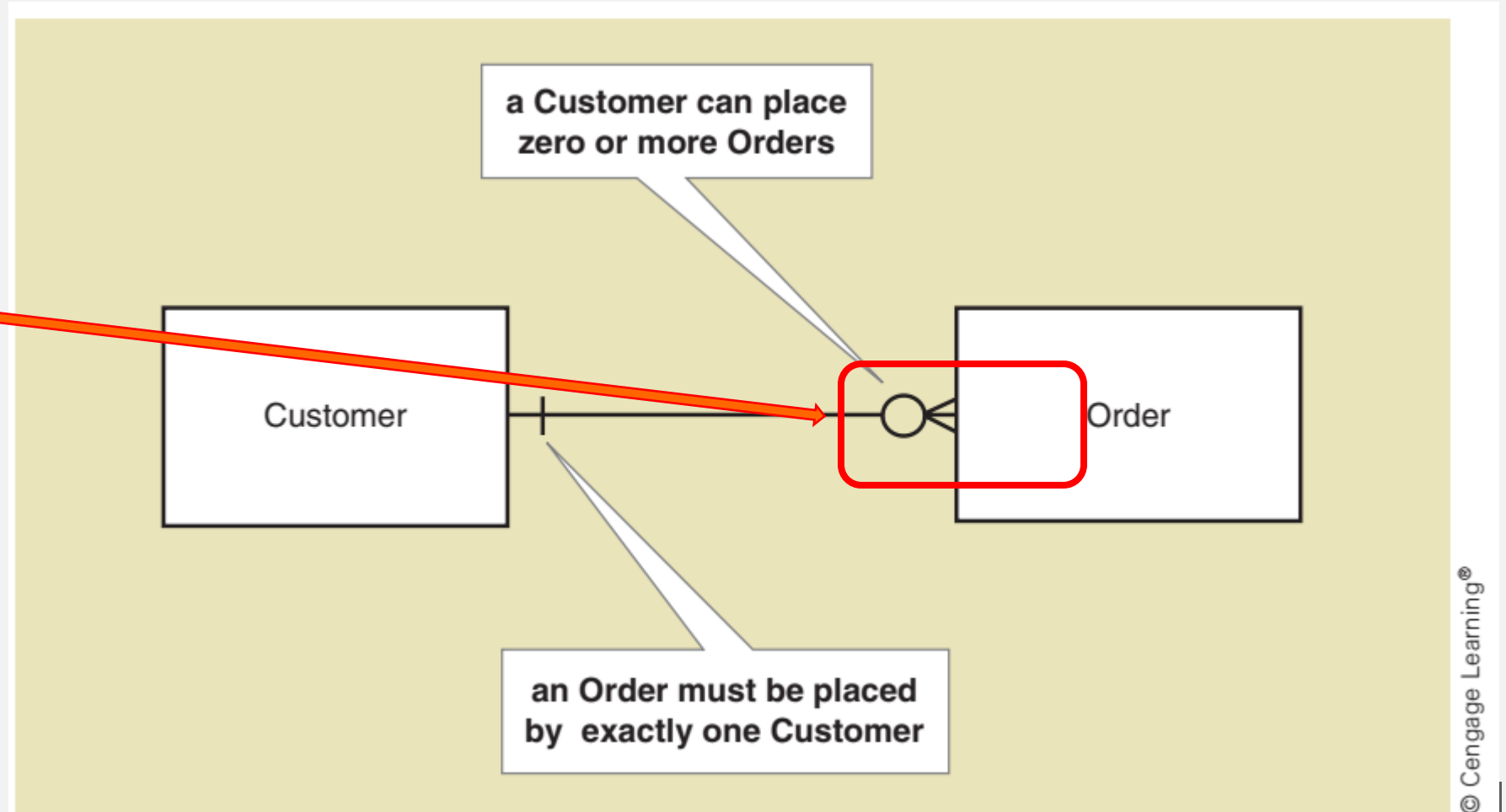
- Binary Association
 - Associations between exactly two different classes
 - ✓ Course Section includes Students
 - ✓ Members join Club
- Unary Association (recursive)
 - Associations between **two instances** of the **same class**
 - ✓ Person married to person
 - ✓ Part is made using parts
- Ternary Association (three)
- N-ary Association (between n)

ERD - ENTITY-RELATIONSHIP DIAGRAMS

- ERDs have been used for many years to develop data models that are used in **database** development
- ERD is a picture that shows the information that is created, stored, and used by a business system.
- An analyst can read an ERD to discover the individual pieces of information in a system and how they are organized and related to each other
- The term for “things” in ERD models is **data entities**
- ERD models are **NOT UML models** and do not use standard UML notation
- ERD models are not as expressive as UML models
 - They do not model generalization/specialization well
 - They do not model whole/part well

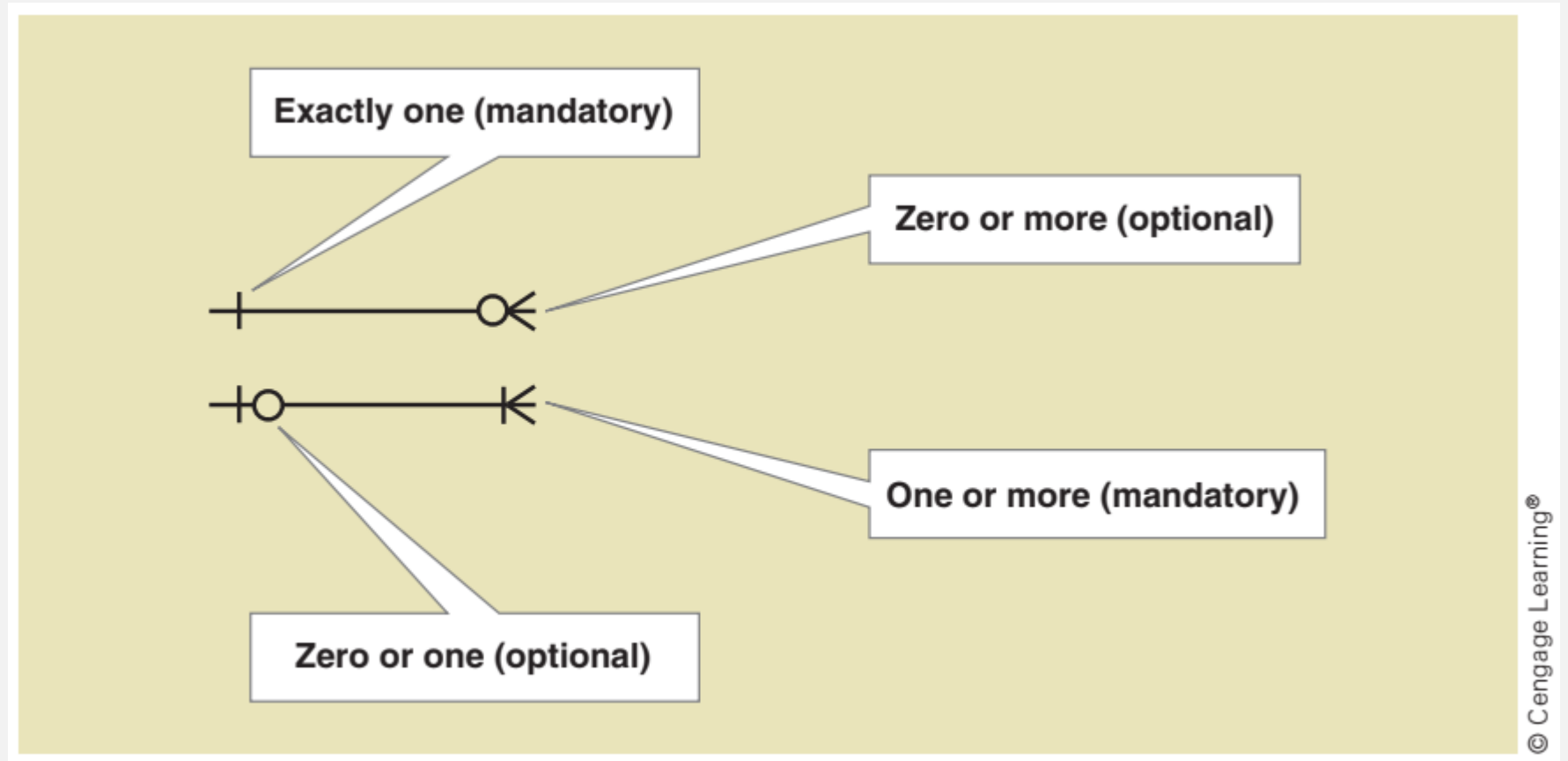
ERD - EXAMPLE OF ERD NOTATION

ERD Models
normally use
“**crows feet**”
notation to
show
cardinality



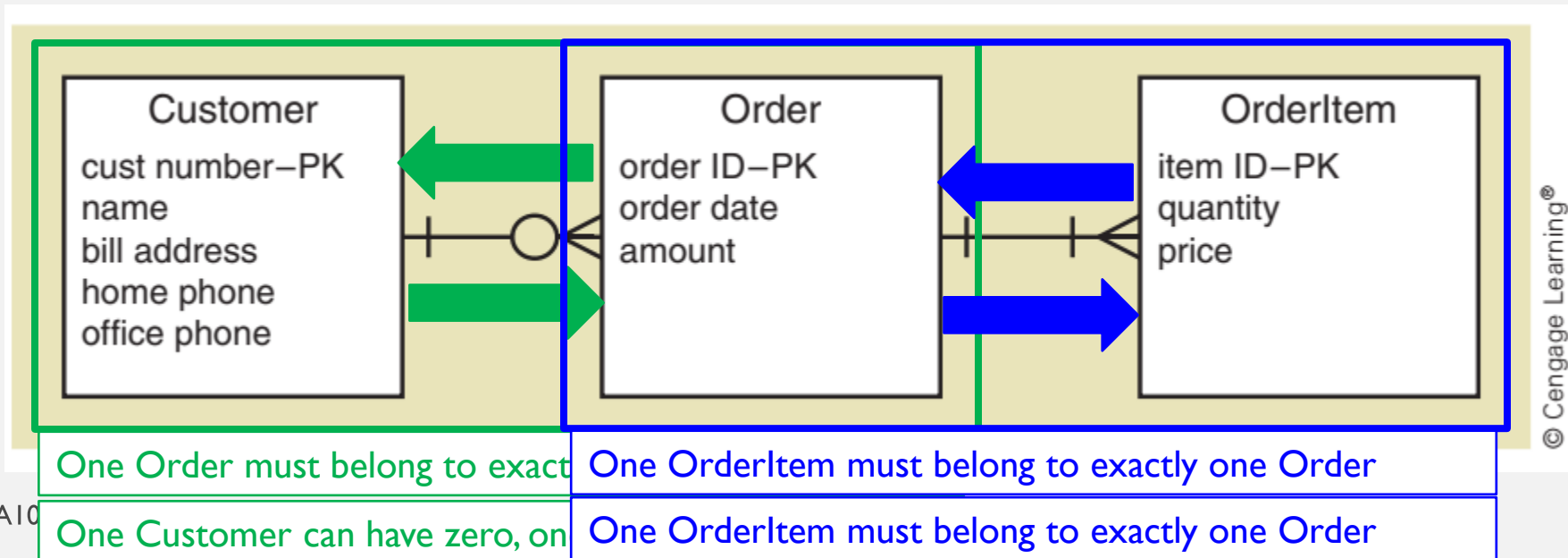
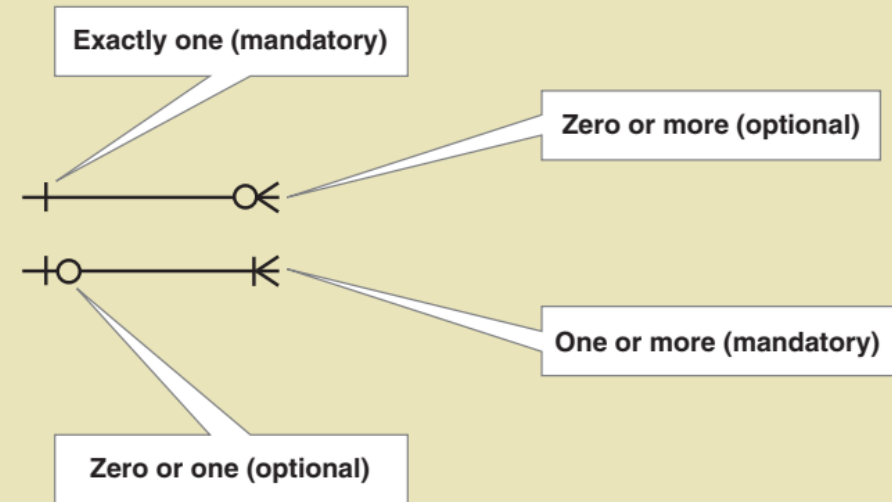
ERD - ERD CARDINALITY SYMBOLS

Examples of
crows feet
notation for
various
cardinalities

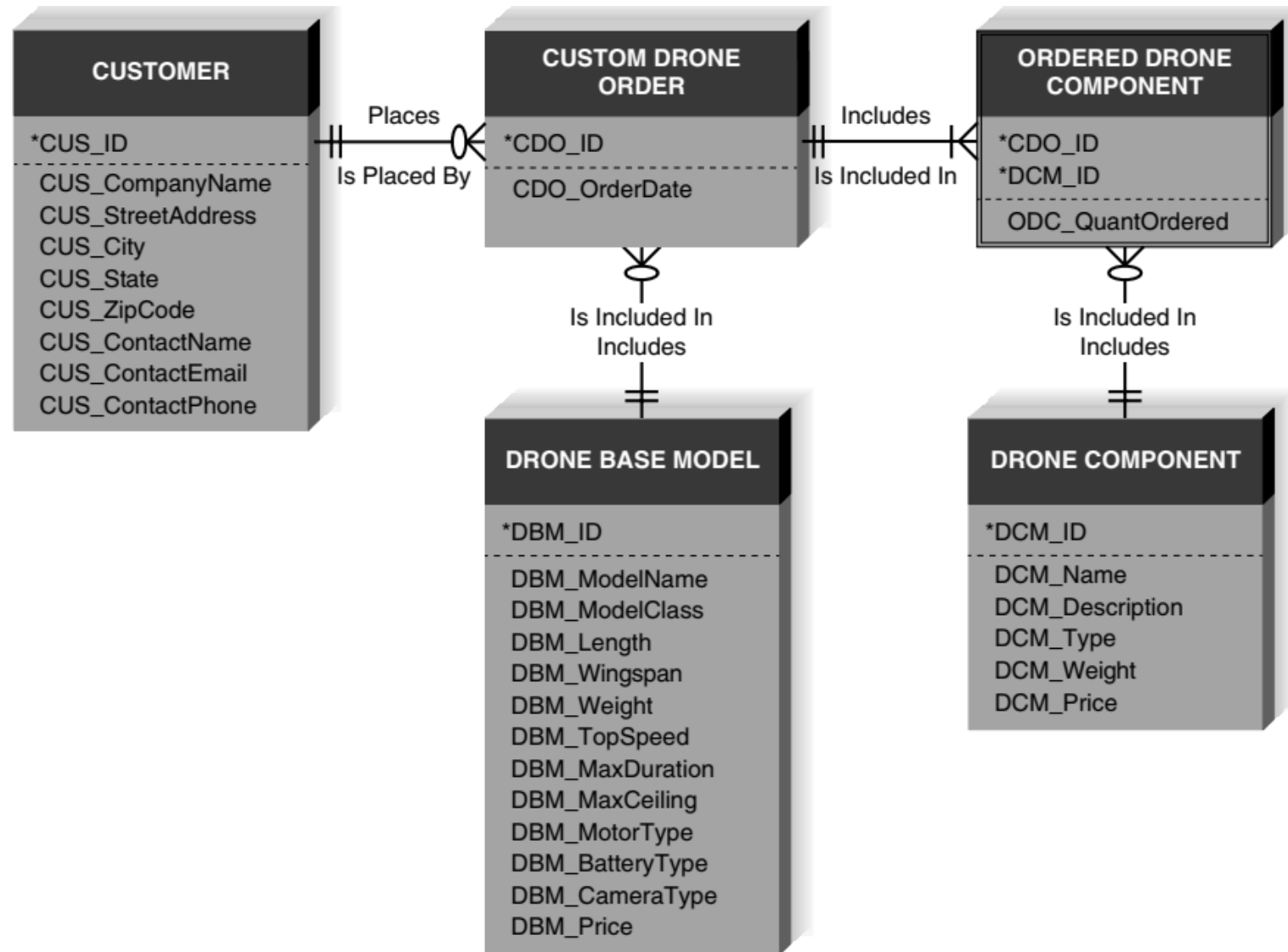


ERD - EXPANDED

- E R D with cardinalities and attri
- There are several different notat models
- This notation places attributes within data entities

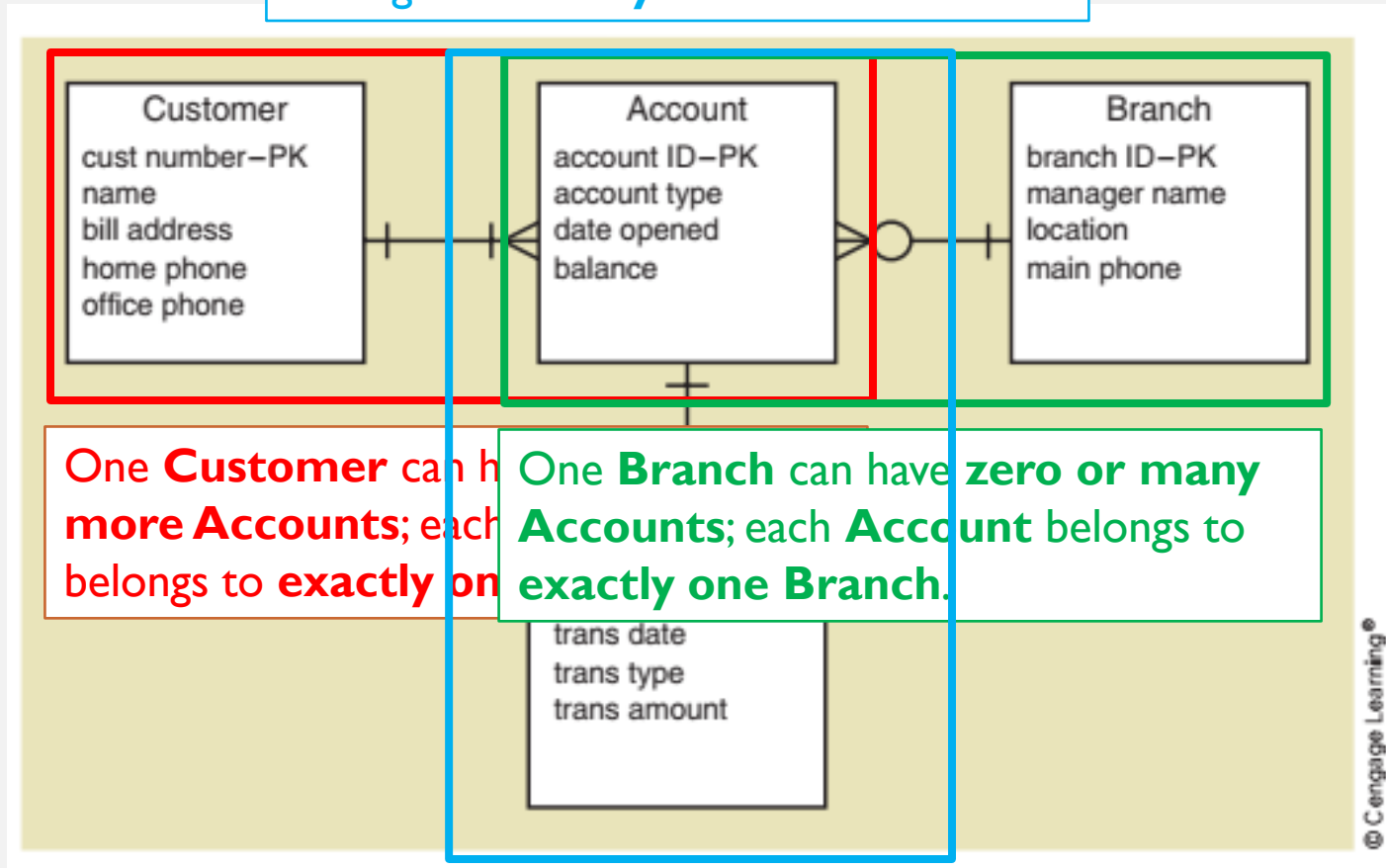


ERD – EXAMPLES OF SALES SYSTEM ERD



ERD ANALYSIS FOR A BANK

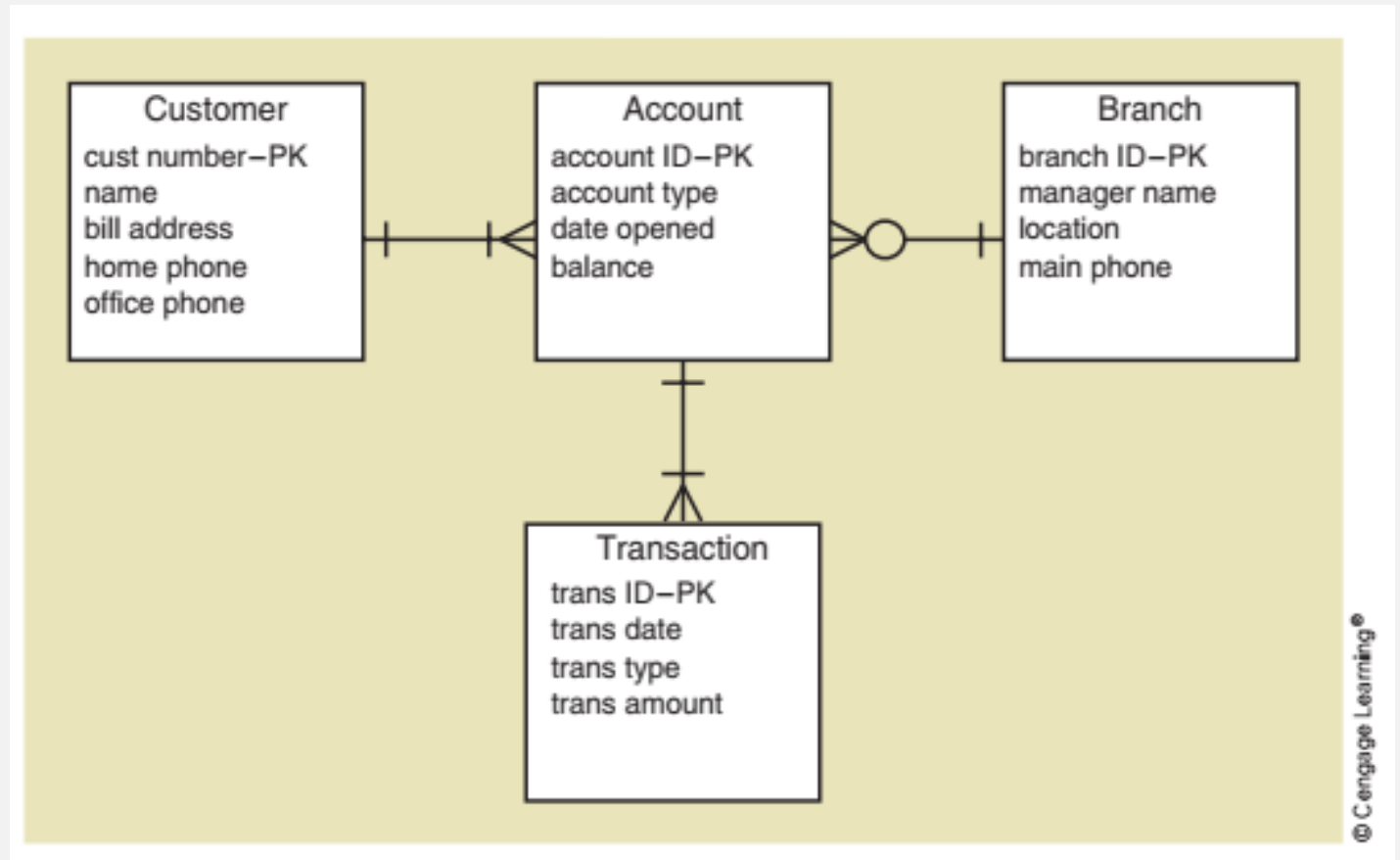
One **Account** can have **one or more Transactions**; each **Transaction** belongs to **exactly one Account**.



ERD - AN ERD FOR A BANK

Class Activity #1

- What are the key fields?
- How many accounts can a customer have?
- How many branches can a customer be assigned to?
- How many customers can a branch have?



RELATIONAL DATABASES

- A relational database management system (RDBMS) is a DBMS that organizes stored data into structures called tables or relations.
- Relational database tables are like conventional tables; that is, they are **2D** data structures of **columns** and **rows**.
- A single row of a table is called a row, tuple, or record, and a column of a table is called an attribute or field.
- A single cell in a table is called an attribute value, field value, or data element.

The screenshot shows a table named 'ProductItem' with the following columns: ProductItemID, Gender, Description, Supplier, Manufacturer, and Picture. The table contains 13 records. Annotations highlight specific parts of the table:

- Field or attribute names:** Points to the column headers.
- One row, tuple, or record:** Points to the entire row for ProductItemID 1244.
- One field or attribute value:** Points to the value 'Man' in the Gender column for ProductItemID 1251.
- One field or attribute and its values:** Points to the entire Description column.

ProductItemID	Gender	Description	Supplier	Manufacturer	Picture
10564	Both	Super Alpine Performance Skis	K2	K2	
10766	Man	Extreme Ski Boots	Nordica	Nordica	
1244	Man	Casual Chino Trousers	West Coast	Adida	
1245	Man	Fleece Crew Sweatshirt	West Coast	Adida	
1246	Man	Fleece Crew Sweatshirt V-Neck	West Coast	Adida	
1247	Man	Fleece Crew Sweatshirt Zippered	West Coast	Adida	
1248	Man	Solid Color Flannel Shirt	RMO	RMO	
1249	Man	Plaid Flannel Shirt	RMO	RMO	
1250	Man	Polo Shirt	RMO	RMO	
1251	Man	Polo Shirt Zippered	RMO	RMO	
1252	Man	Navigator Jacket	Colorado Supply	North Face	
1253	Man	Navigator Jacket Hooded	Colorado Supply	North Face	
1254	Man	Cotton Thermal Shirt	Colorado Supply	Under Armour	

Source: Microsoft Corporation



TYPES OF KEYS IN RELATIONAL DATABASES

Unique Key

- Each table must have a **unique key**.
- Uniquely identifies each row in the table



Example Table

Customer ID	Name	Email	City
1021	John Doe	Doe	Boston
1053	Jane Smith	jim@gmail.com	
1072	Bob Lee	bob@company.com	

Foreign Key

- Duplicates the **primary** key of a **different** table

Order Table

Order ID	1053	Amount
3503	1053	\$250

Candidate Key

Values that could serve as a unique identifier:

- Email
- Customer ID
- Phone Number



Primary Key

- The key **chosen** by a database designer as the main unique identifier

- ☒ Customer ID
- ☒ Phone Number



Foreign Key

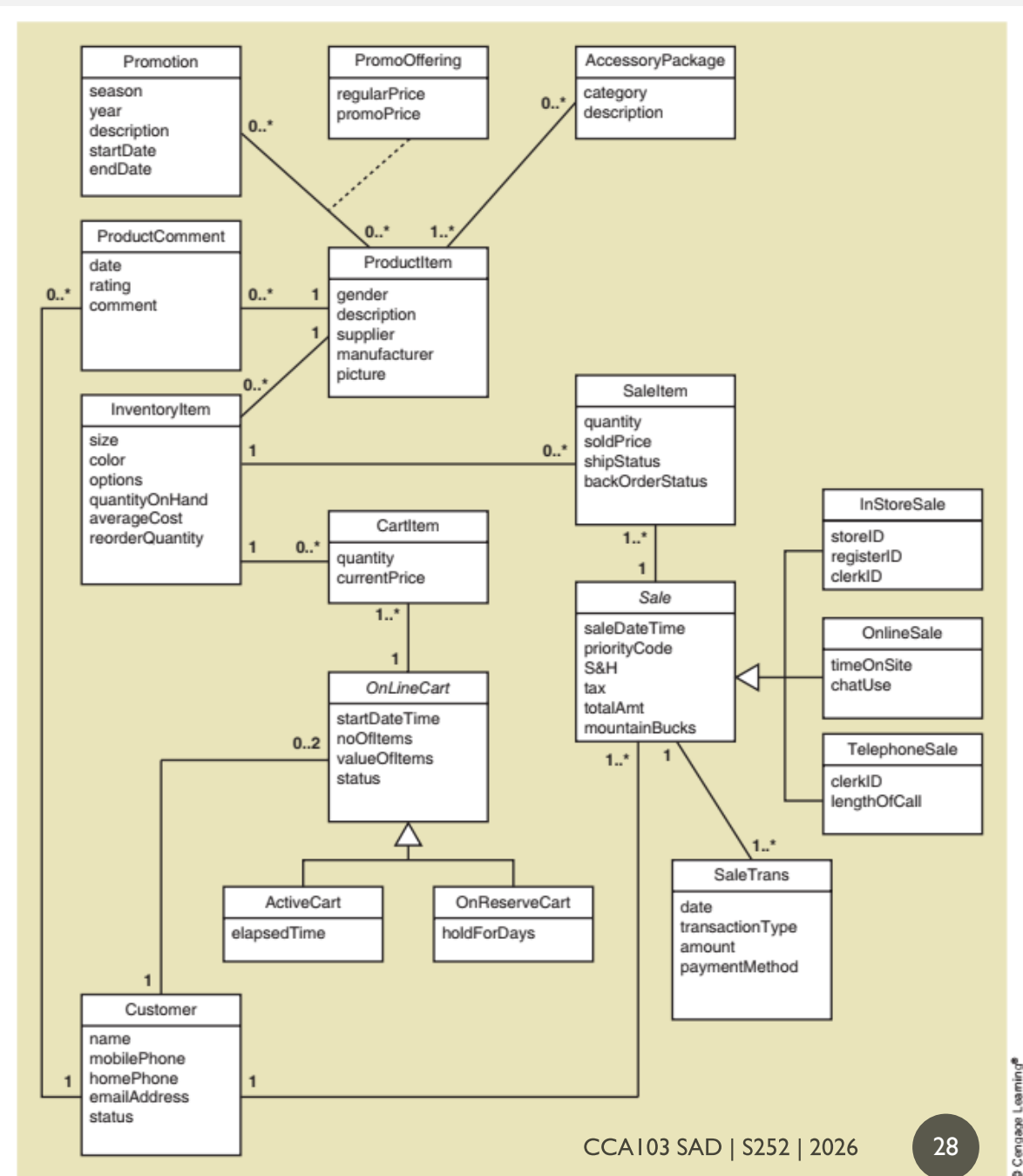
- Duplicates the **primary** key of a **different** table

Order Table

Order ID	Amount
3503	\$250

CREATING TABLES FROM DOMAIN CLASSES #1

- The first step in creating a relational database schema is to **create a table** for each **class** on the **class diagram**.
- To avoid confusion, table and attribute names should match the names used in the class diagram and abbreviations should be avoided.



CREATING TABLES FROM DOMAIN CLASSES #2

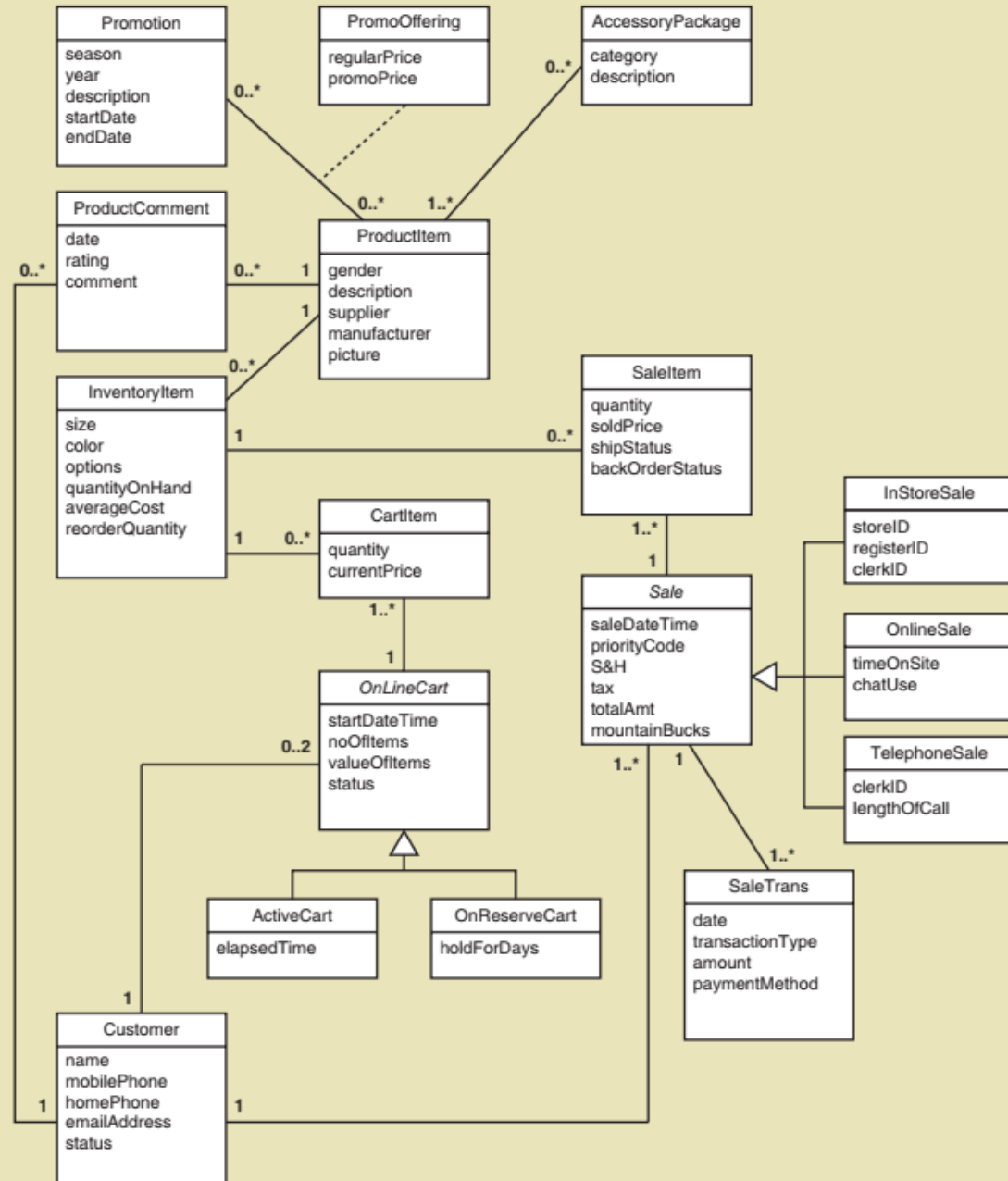


Table	Attributes
AccessoryPackage	Category, Description
CartItem	Quantity, CurrentPrice
Customer	Name, MobilePhone, HomePhone, EmailAddress, Status
InventoryItem	Size, Color, Options, QuantityOnHand, AverageCost, ReorderQuantity
OnlineCart	StartDateTime, NumberOfItems, ValueOfItems, Status, ElapsedTime, HoldForDays
ProductComment	Date, Rating, Comment
ProductItem	Gender, Description, Supplier, Manufacturer, Picture
PromoOffering	RegularPrice, PromoPrice
Promotion	Season, Year, Description, StartDate, EndDate
Sale	SaleDateTime, PriorityCode, ShippingAndHandling, Tax, TotalAmount, MountainBucks, StoreID, RegisterID, ClerkID, TimeOnSite, ChatUse, LengthOfCall
SaleItem	Quantity, SoldPrice, ShipStatus, BackOrderStatus
SaleTransaction	Date, TransactionType, Amount, PaymentMethod

CHOOSING PRIMARY KEYS #3

After creating a table, we choose a primary key for it.

- If the table already has a field, or a combination of fields, that is always unique, we can use it as the primary key.
- If there is no unique field, we create a new one to identify each record.

This new field is usually given a clear name, such as ID, Code, or Number, for example, StudentID or OrderID.

Table	Attributes
AccessoryPackage	AccessoryPackageID , AccessoryCategory, Description
CartItem	CartItemID , Quantity, CurrentPrice
Customer	AccountNumber , Name, MobilePhone, HomePhone, EmailAddress, Status
InventoryItem	InventoryItemID , Size, Color, Options, QuantityOnHand, AverageCost, ReorderQuantity
OnlineCart	OnlineCartID , StartDateTime, NumberOfItems, ValueOfItems, Status, ElapsedTime, HoldForDays
ProductComment	ProductCommentID , Date, Rating, Comment
ProductItem	ProductItemID , Gender, Description, Supplier, Manufacturer, Picture
PromoOffering	PromoOfferingID , RegularPrice, PromoPrice
Promotion	PromotionID , Season, Year, Description, StartDate, EndDate
Sale	SaleID , SaleDateTime, PriorityCode, ShippingAndHandling, Tax, TotalAmount, MountainBucks, StoreID, RegisterID, ClerkID, TimeOnSite, ChatUse, LengthOfCall
SaleItem	SaleItemID , Quantity, SoldPrice, ShipStatus, BackOrderStatus
SaleTransaction	SaleTransactionID , Date, TransactionType, Amount, PaymentMethod

REPRESENTING ASSOCIATIONS

- **Associations** are represented within a relational **database** by **foreign keys**.
- The choice of which foreign keys to place in which tables depends on the type of association being represented.
- You will deal with the generalization/ specialization associations in a later step.
- The rules for representing one-to-many and many-to-many associations are as follows

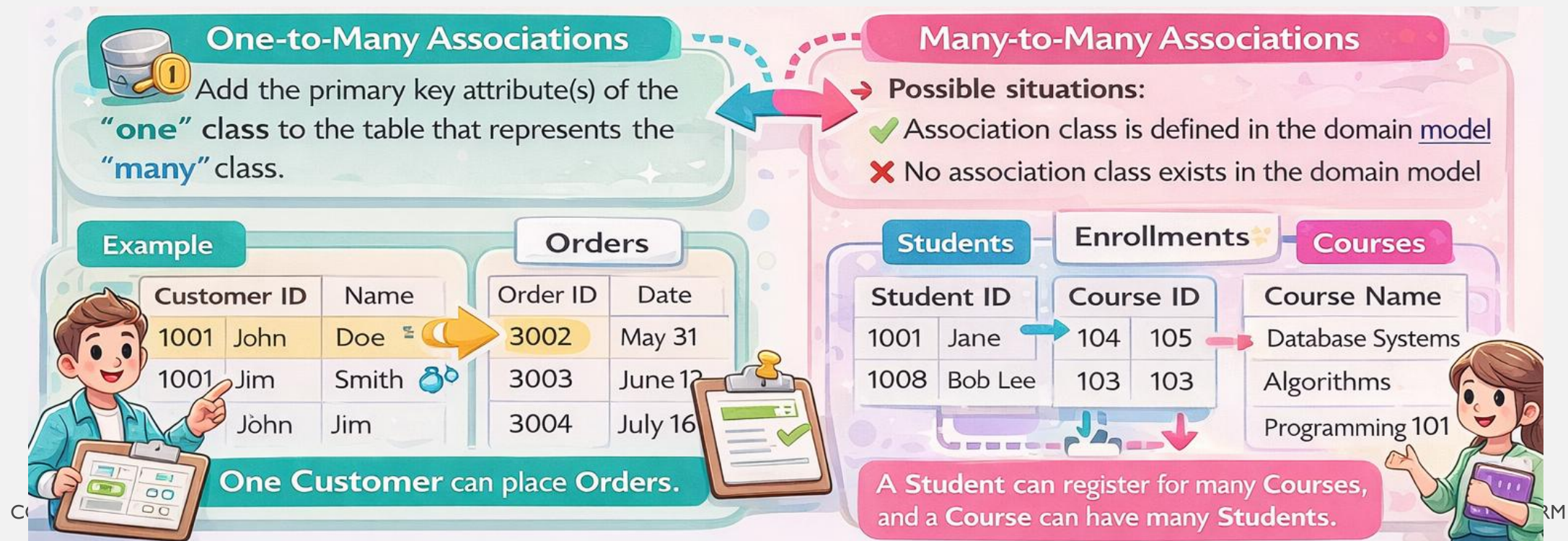


Table	Attributes
AccessoryPackage	AccessoryPackageID , AccessoryCategory, Description
CartItem	CartItemID , <i>InventoryItemID</i> , <i>OnlineCartID</i> , Quantity, CurrentPrice
Customer	AccountNumber , Name, MobilePhone, HomePhone, EmailAddress, Status
InventoryItem	InventoryItemID , <i>ProductItemID</i> , Size, Color, Options, QuantityOnHand, AverageCost, ReorderQuantity
OnlineCart	OnlineCartID , <i>CustomerAccountID</i> , StartDateTime, NumberOfItems, ValueOfItems, Status, ElapsedTime, HoldForDays
ProductComment	ProductCommentID , <i>ProductItemID</i> , <i>CustomerAccountNumber</i> , Date, Rating, Comment
ProductItem	ProductItemID , Gender, Description, Supplier, Manufacturer, Picture
PromoOffering	PromoOfferingID , RegularPrice, PromoPrice
Promotion	PromotionID , Season, Year, Description, StartDate, EndDate
Sale	SaleID , <i>CustomerAccountNumber</i> , SaleDateTime, PriorityCode, ShippingAndHandling, Tax, TotalAmount, MountainBucks, StoreID, RegisterID, ClerkID, TimeOnSite, ChatUse, LengthOfCall
SaleItem	SaleItemID , <i>InventoryItemID</i> , <i>SaleID</i> , Quantity, SoldPrice, ShipStatus, BackOrderStatus
SaleTransaction	SaleTransactionID , <i>SaleID</i> , Date, TransactionType, Amount, PaymentMethod

ASSOCIATIONS ONE-TO-MANY

- One-to-many associations are represented by adding foreign key attributes (shown in *italic*)
- Each foreign key represents a single association between the table containing the foreign key and the table that uses that same key as its primary key.
- For example, the attribute *CustomerAccountNumber* was added to the Sale table as a foreign key representing the one-to-many association between the Customer and Sale classes.
- The foreign key *SaleID* was added to the SaleTransaction table to represent the one-to-many association between Sale and SaleTransaction.

Table	Attributes
AccessoryPackage	AccessoryPackageID , AccessoryCategory, Description
AccessoryPackageContents	<i>AccessoryPackageID</i> , <i>ProductItemID</i>
CartItem	<i>InventoryItemID</i> , <i>OnlineCartID</i> , Quantity, CurrentPrice
Customer	AccountNumber , Name, MobilePhone, HomePhone, EmailAddress, Status
InventoryItem	InventoryItemID , <i>ProductItemID</i> , Size, Color, Options, QuantityOnHand, AverageCost, ReorderQuantity
OnlineCart	OnlineCartID , <i>CustomerAccountID</i> , StartDateTime, NumberOfItems, ValueOfItems, Status, ElapsedTime, HoldForDays
ProductComment	ProductCommentID , <i>ProductItemID</i> , <i>CustomerAccountNumber</i> , Date, Rating, Comment
ProductItem	ProductItemID , Gender, Description, Supplier, Manufacturer, Picture
PromoOffering	<i>PromotionID</i> , <i>ProductItemID</i> , RegularPrice, PromoPrice
Promotion	PromotionID , Season, Year, Description, StartDate, EndDate
Sale	SaleID , <i>CustomerAccountNumber</i> , SaleDateTime, PriorityCode, ShippingAndHandling, Tax, TotalAmount, MountainBucks, StoreID, RegisterID, ClerkID, TimeOnSite, ChatUse, LengthOfCall
SaleItem	<i>InventoryItemID</i> , <i>SaleID</i> , Quantity, SoldPrice, ShipStatus, BackOrderStatus
SaleTransaction	SaleTransactionID , <i>SaleID</i> , Date, TransactionType, Amount, PaymentMethod

ASSOCIATIONS MANY-TO-MANY

- PromoOffering table modified to represent the many-to-many association between Product and Promotion
- The primary key of PromoOffering becomes the combination of PromotionID and ProductItemID.
- The old primary key PromoOfferingID is discarded.
- The two attributes that make up the primary key are also foreign keys and are displayed in boldface and italic to indicate their dual status.
- PromotionID is a foreign key from the Promotion table, and ProductItemID is a foreign key from the ProductItem table.

REPRESENTING CLASSIFICATION HIERARCHIES

- Just as a specialized class inherits the data and methods of a generalized class, a table representing a specialized class inherits some or all of its data from the table representing its generalized class.
- This inheritance can be represented in multiple ways, including the following:
 - ✓ **Method #1: Combining** all the tables into a single table containing the superset of all classes (Put parent data and child data together in **one table**) ~ **1 Table stores everything**
 - ✓ **Method #2:** Using separate tables to represent the child classes, and using the primary key of the parent class table as the primary key of the child class tables (1 table for parent + 1 table for child) ~ **Common data stays in the parent table, special data stays in child tables.**
 - ✓ Some combination of the previous two methods

FIGURE 9-11 Specialization classes of Sale and OnlineCart included in the primary tables

Table	Attributes
AccessoryPackage	AccessoryPackageID , AccessoryCategory, Description
AccessoryPackageContents	AccessoryPackageID, ProductItemID
CartItem	InventoryItemID, OnlineCartID , Quantity, CurrentPrice
Customer	AccountNumber , Name, MobilePhone, HomePhone, EmailAddress, Status
InventoryItem	InventoryItemID, ProductItemID , Size, Color, Options, QuantityOnHand, AverageCost, ReorderQuantity
OnlineCart	OnlineCartID, CustomerAccountID , StartDateTime, NumberOfItems, ValueOfItems, Status, ElapsedTime, HoldForDays
ProductComment	ProductCommentID, ProductItemID, CustomerAccountNumber , Date, Rating, Comment
ProductItem	ProductItemID , Gender, Description, Supplier, Manufacturer, Picture
PromoOffering	PromotionID, ProductItemID , RegularPrice, PromoPrice
Promotion	PromotionID , Season, Year, Description, StartDate, EndDate
Sale	SaleID, CustomerAccountNumber , SaleDateTime, PriorityCode, ShippingAndHandling, Tax, TotalAmount, MountainBucks, <u>StoreID</u> , <u>RegisterID</u> , <u>ClerkID</u> , <u>TimeOnSite</u> , <u>ChatUse</u> , <u>LengthOfCall</u>
SaleItem	InventoryItemID, SaleID , Quantity, SoldPrice, ShipStatus, BackOrderStatus
SaleTransaction	SaleTransactionID, SaleID , Date, TransactionType, Amount, PaymentMethod

Specialise class

StoreID, RegisterID, ClerkID = physicalStore

saleTimeOnSite, ChatUse, LengthOfCall =phoneSale

DATABASE NORMALIZATION

- A good database starts with an accurate and complete data model; otherwise, the database may become incorrect or inefficient.
- It is important to do a final review of the database design to make sure the database is well-formed and correct.
- A correct database should be normalized, because normalization helps improve accuracy, consistency, and performance.
- A normalized relational database schema has three important characteristics:

1. Allows flexibility in implementing future data model changes
2. Contains a minimum of redundant data
3. Prevents insertion, deletion, and update anomalies



DATABASE NORMALIZATION

★ REDUCE DATA REDUNDANCY

- To reduce **data redundancy**, avoid unnecessary repeated data



DATA REDUNDANCY



✓ PREVENT DATA ANOMALIES

- Helps prevent data **insertion**, updating, and deletion errors



LEVELS OF NORMALIZATION

Data organized into levels:

1NF | 2NF | 3NF | 4NF | BCNF



LEVELS OF NORMALIZATION



Data organized into levels:

1NF | 2NF | 3NF | 4NF | BCNF



AIM FOR 3NF

- Third Normal Form (3NF) is a good standard for most business systems

3NF

Third Normal Form (3NF) is a good standard for most business systems.





DATABASE NORMALIZATION

- FIRST NORMAL FORM (1NF)

Table is in 1NF if...

- Every field contains **only one value**

✓ All attribute values must be **atomic**



Multivalued Attribute

- All attributes must be **atomic**; no multiple values in one field

Customer ID		Name	Contact Number
1001	John	Doe	555-1234, 555-5678
1001	John	Doe	555-1234



Repeating Columns

- No repeating **columns** allowed



1001	John	Email	Email
1001	Doe	john@gmail.com	



Customer ID Emails

Customer ID	Name	Emails
1001	John	john@gmail.com
1001	John	doe@yahoo.com



A table is in first normal form (1NF) if every field contains only one value.



DATABASE NORMALIZATION - 1NF

FIGURE 9-13(a) Rows with multivalued attribute Dependents

SSN	Name	Department	Salary	Dependents
111-22-3333	Mary Smith	Accounting	40,000	John, Alice, Dave
222-33-4444	Jose Pena	Marketing	50,000	---
333-44-5555	Frank Collins	Production	35,000	Jan, Julia

FIGURE 9-13(b) Rows with varying number of Dependent columns

SSN	Name	Department	Salary	Dependent	Dependent	Dependent
111-22-3333	Mary Smith	Accounting	40,000	John	Alice	Dave
222-33-4444	Jose Pena	Marketing	50,000			
333-44-5555	Frank Collins	Production	35,000	Jan	Julia	

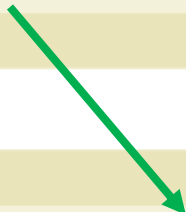
Figure 9-13(a) illustrates the first approach, and Figure 9-13(b) illustrates the second.

1NF prohibits attributes such as Dependent, which is shown in Figure 9-13.

DATABASE NORMALIZATION - 1NF

- To correct this problem, the employee table must be **divided** into **two** tables.
- The first table will have the columns of SSN, Name, Department, and Salary.
- Another table, with a newly invented key, will have columns SSN and Dependent.
- Notice in the second table that each row has a unique key and also has a pairwise matching of SSNs and Dependents.

SSN	Name	Department	Salary
111-22-3333	Mary Smith	Accounting	40,000
222-33-4444	Jose Pena	Marketing	50,000
333-44-5555	Frank Collins	Production	35,000



RecordID	SSN	Dependent
1	111-22-3333	John
2	111-22-3333	Alice
3	111-22-3333	Dave
4	333-44-5555	Jan
5	333-44-5555	Julia

DATABASE NORMALIZATION - SECOND NORMAL FORM (2NF)

Table is in **2NF** if...

- It is in **1NF** ✓
- Each non-key attribute is **fully dependent** on the entire primary key.

✓ **2NF Compliant**

🔑 **Orders Table**

1001	Product ID
1001	Keyboard
1002	Mouse



🔑 **Products Table**

Order ID	Product Name	Unit Price
P01	Keyboard	\$50
P02	Mouse	\$20

⚠ **Violation of 2NF**

- Non-key attribute (Unit Price) is dependent only on part of the primary key.



Examine each non-key attribute's dependency on all parts of the primary key

✓ **2NF Compliant**

🔑 **Orders Table**

1001	P01	P02
1002	P01	P02



🔑 **Products Table**

P01	Keyboard
P02	Mouse

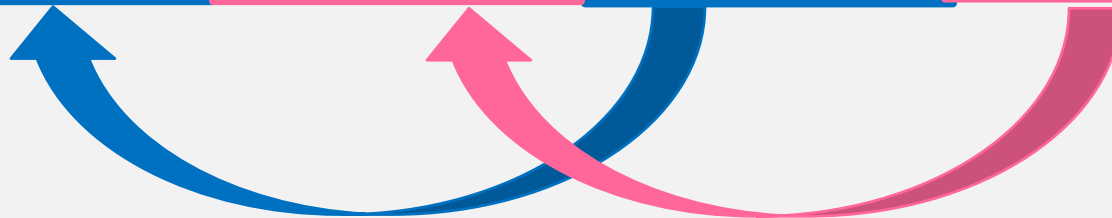


A table is in **second normal form (2NF)** if each non-key attribute is **fully dependent** on the entire primary key.

2NF PROBLEM

PK = StudentID & CourseCode

StudentID	CourseCode	StudentName	CourseName	Grade
S001	CSCI01	Ali	Database	A
S001	CSCI02	Ali	Programming	B
S002	CSCI01	Sara	Database	A-



StudentName depends only on **StudentID**
CourseName depends only on **CourseCode**

2NF SOLUTION

Student Table

StudentID	StudentName
S001	Ali
S002	Sara

Course Table

CourseCode	CourseName
CSCI01	Database
CSCI02	Programming

Grade table

StudentID	CourseCode	Grade
S001	CSCI01	A
S001	CSCI02	B
S002	CSCI01	A-



DATABASE NORMALIZATION - - THIRD NORMAL FORM (3NF)

Table is in 3NF if...

- It is in **2NF** ✓
- No non-key attribute is functionally dependent on any other non-key attribute.



✓ 3NF Compliant

🔑 Orders Table

1001	Cust ID
1001	John Doe
1002	Jane Smith

🔑 Customers Table

Cust ID	Cust Name	City
101	Beverly Hills	90210
102	New York	10001



⚠️ VIOLATION OF 3NF

Order ID	Cust ID	Zip Code
1001	John Doe	90210
1002	Jane Smith	10001



- Non-key attribute (City) is dependent on another non-key attribute (Zip Code).

✓ 3NF Compliant

🔑 Cust ID	Cust Name	Zip Code	City
101	John Doe	90210	Beverly Hills
102	Jane Smith	10001	New York

- No non-key attribute is dependent on any other non-key attribute.



3NF EXAMPLE

PK = StudentID

StudentID	CourseCode	StudentName	CourseName	Grade
S001	CSC101	Ali	Database	A
S001	CSC102	Ali	Programming	B
S002	CSC101	Sara	Database	A-

CourseName depends only on **CourseCode**

THE DOMAIN MODEL CLASS DIAGRAM

Class

- A type of classification used to describe a collection of objects

Domain Class

- Classes that describe objects in the problem domain

Class Diagram

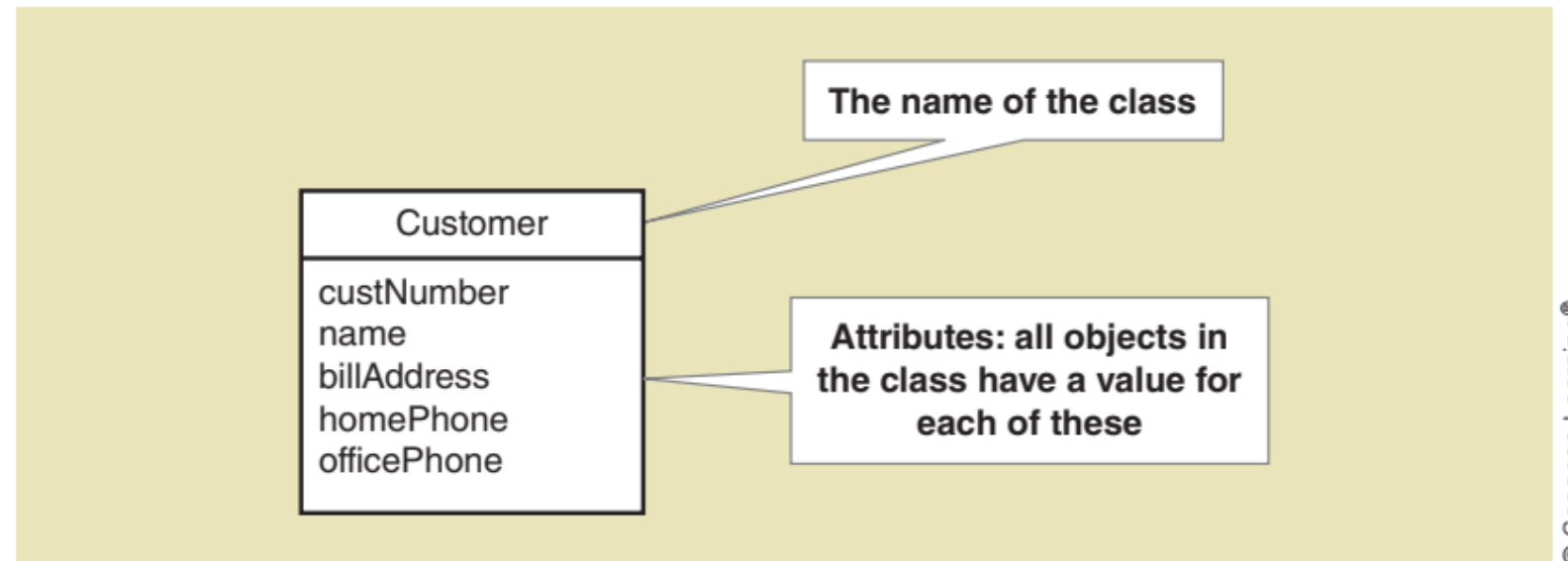
- A UML diagram that shows classes with attributes and associations (plus methods if it models software classes)

Domain Model Class Diagram

- A class diagram that only includes classes from the problem domain, not software classes so no methods

DOMAIN CLASS - NOTATION

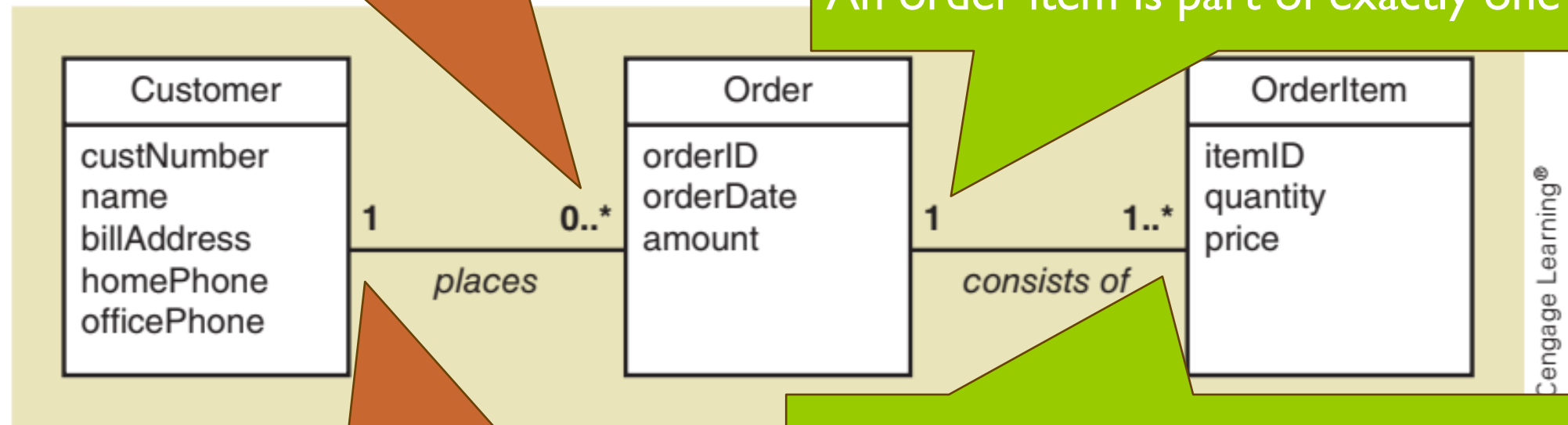
- Domain class a name and attributes (no methods)
- **Class name** is always **capitalized**
- Attribute names are not capitalized and use **camelback notation** (words run together and second word is capitalized)
- Compound class names also use camelback notation



DOMAIN CLASS - A SIMPLE DOMAIN MODEL CLASS DIAGRAM

A customer places zero or more orders

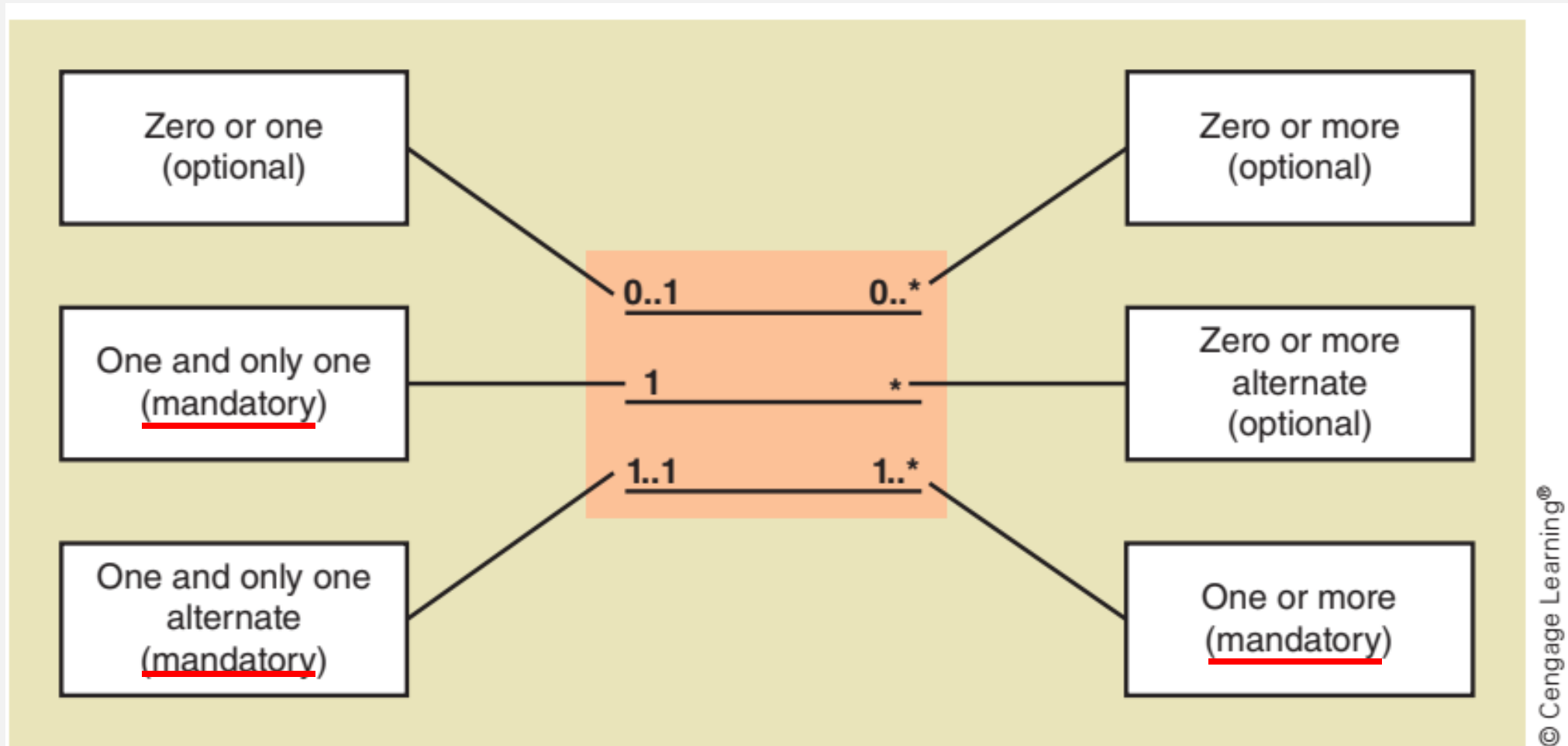
An order item is part of exactly one order



An order consists of one or more order items

An order is placed by exactly one customer

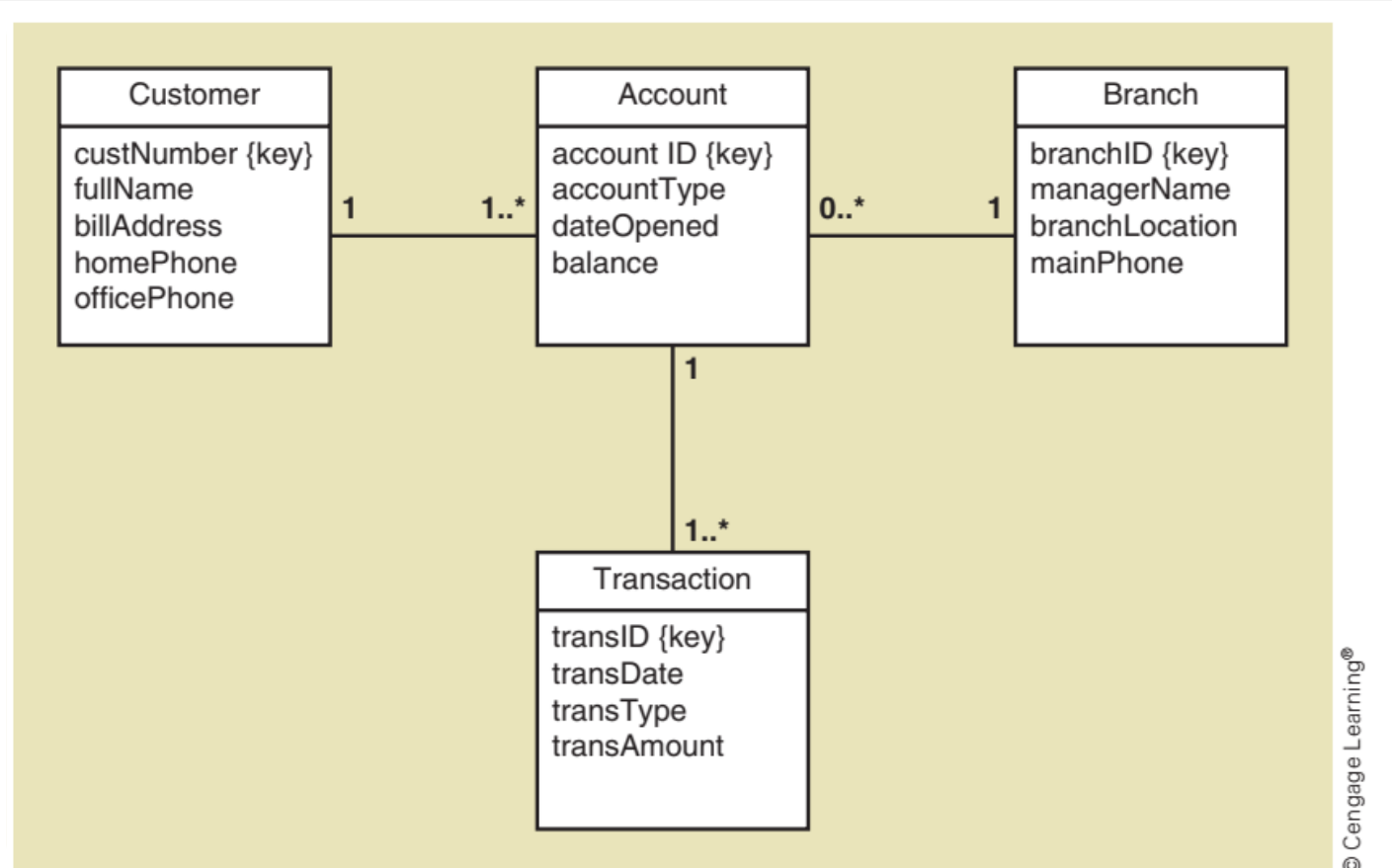
DOMAIN CLASS - UML NOTATION FOR MULTIPLICITY



DOMAIN CLASS –EXAMPLE #I

Bank with many branches, as shown previously in ERD

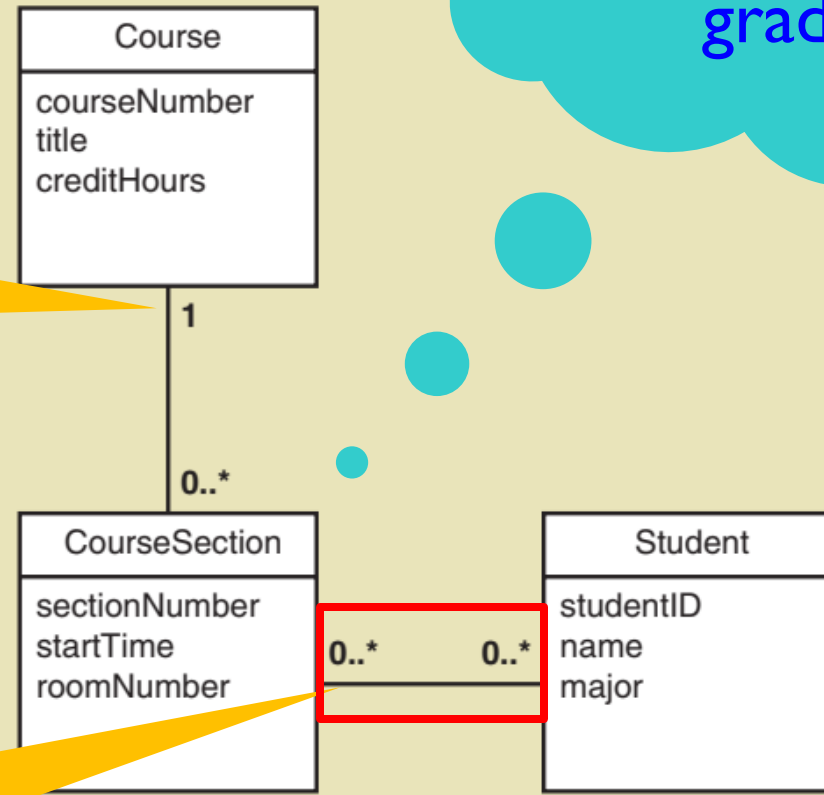
- Note notation for the key
- Note the precise notation for the attributes (camelback)
- Note the multiplicity notation



DOMAIN CLASS –EXAMPLE #1

Course Enrollment at a University

At a university, courses are offered as course sections, and a student enrolls in many course sections.



many-to-many association

Each course section contains many students. Therefore, the association between CourseSection and Student is many-to-many.

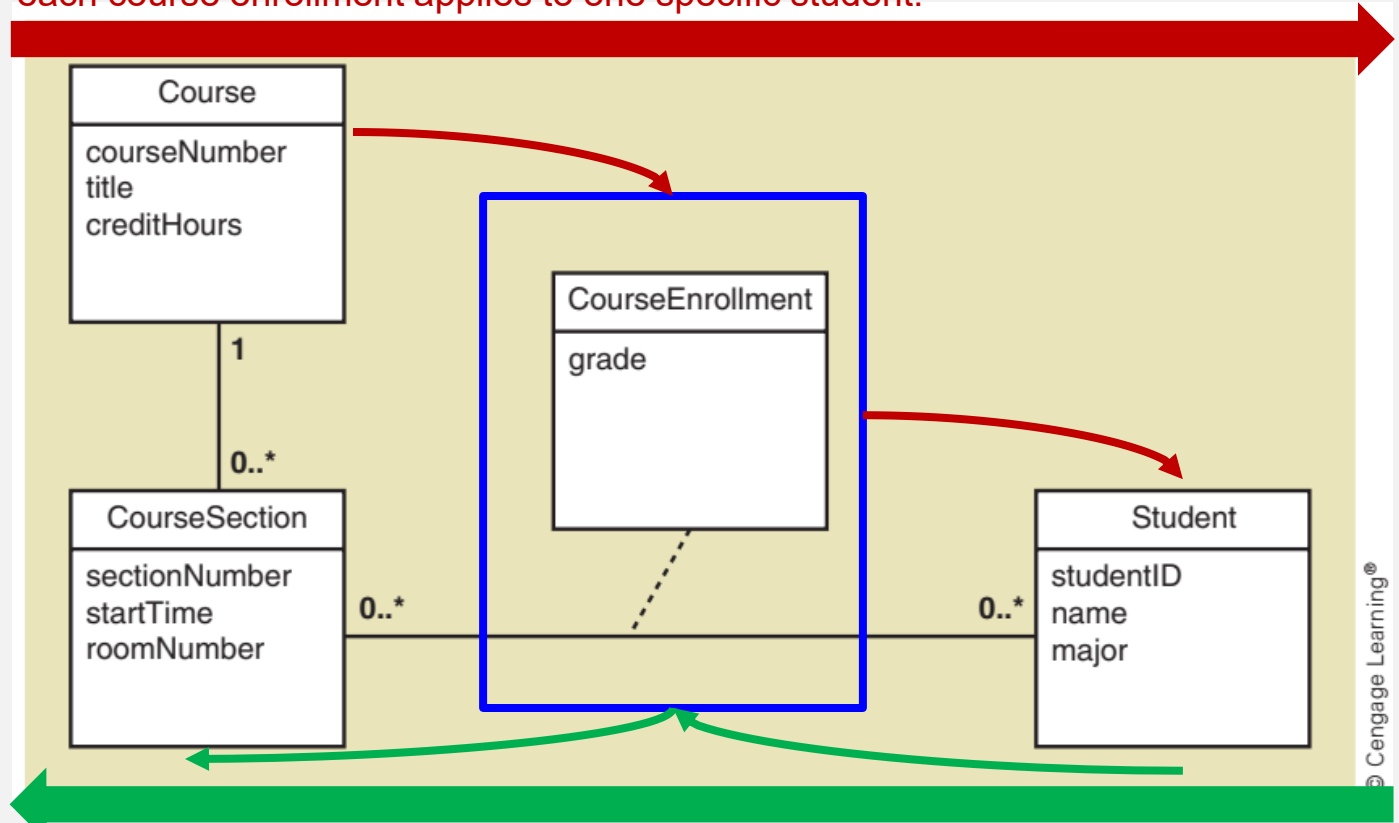
Problem
How/where to capture student grades?

DOMAIN CLASS –EXAMPLE #2: REFINED COURSE ENROLLMENT MODEL WITH AN ASSOCIATION CLASS COURSEENROLLMENT

Association class

An association that is treated as a class in a many to many association because it has attributes that need to be remembered (such as grade)

One course section has many course enrollments—each with its own grade—and each course enrollment applies to one specific student.



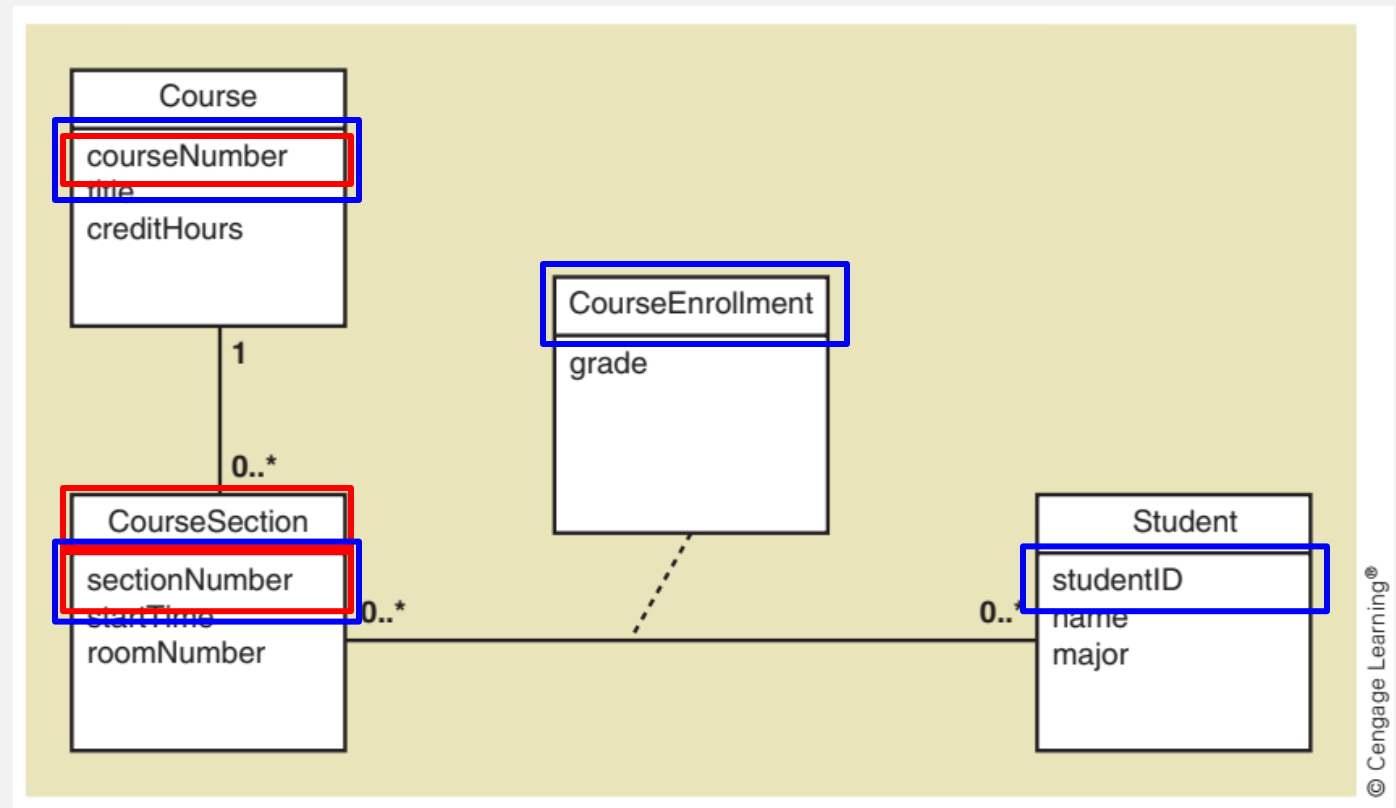
One student has many course enrollments—each with its own grade—and each course enrollment applies to one specific course section.

ASSOCIATION CLASS PROPERTIES

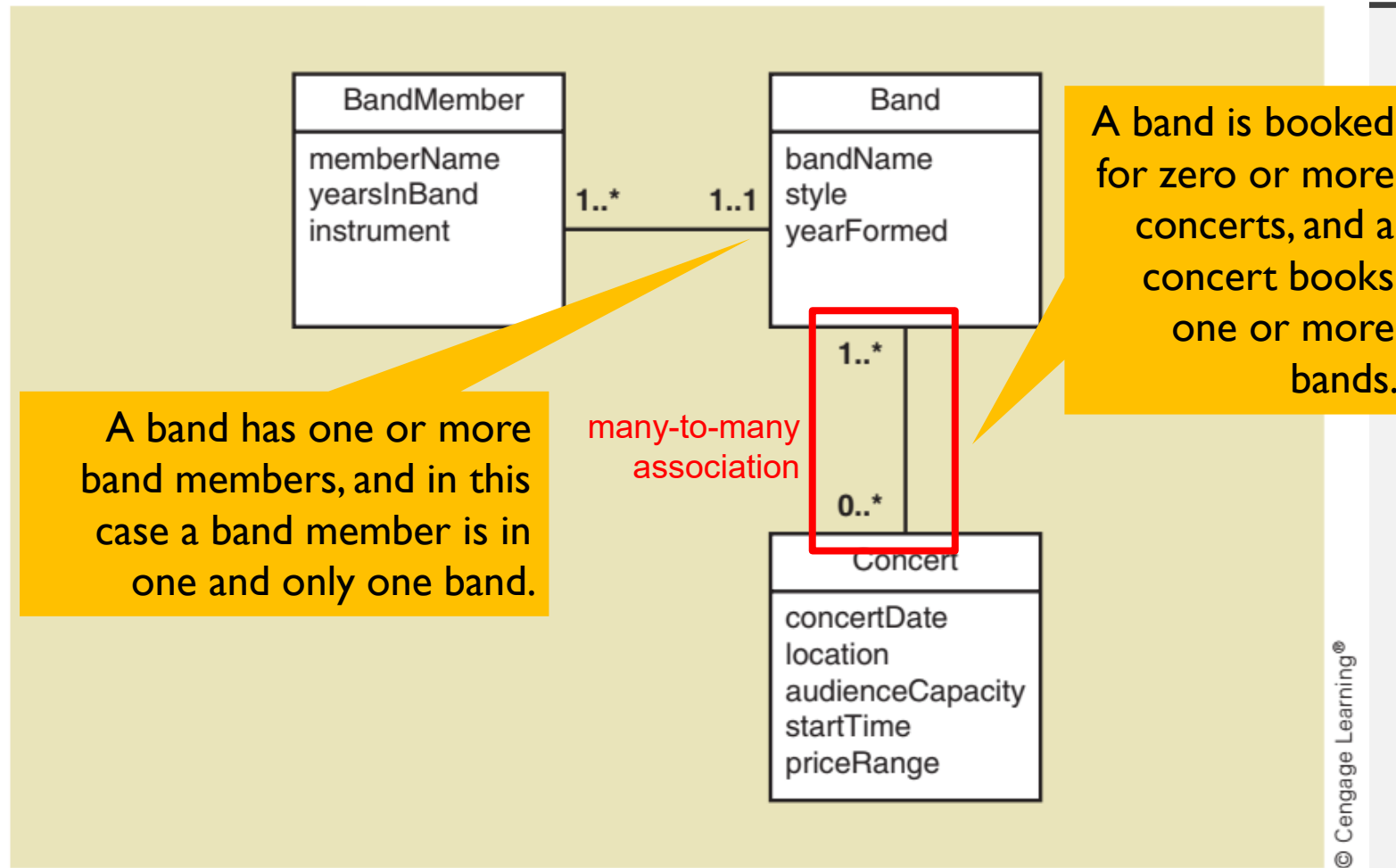
- The association class is the same “thing” as the association itself
- The unique identifier (**key**) for the association class is the concatenation of the keys of the attached classes
- In the previous example, the key for CourseSection is CourseNumber+SectionNumber
- Hence, the key for CourseEnrollment is CourseNumber+SectionNumber+Student ID

Note: If more information is required to uniquely identify instances of the association class, then the model is incorrect, i.e., if the key cannot be formed by the concatenation of the endpoint keys, it is in error.

CCA103 SAD | S252 | 2026



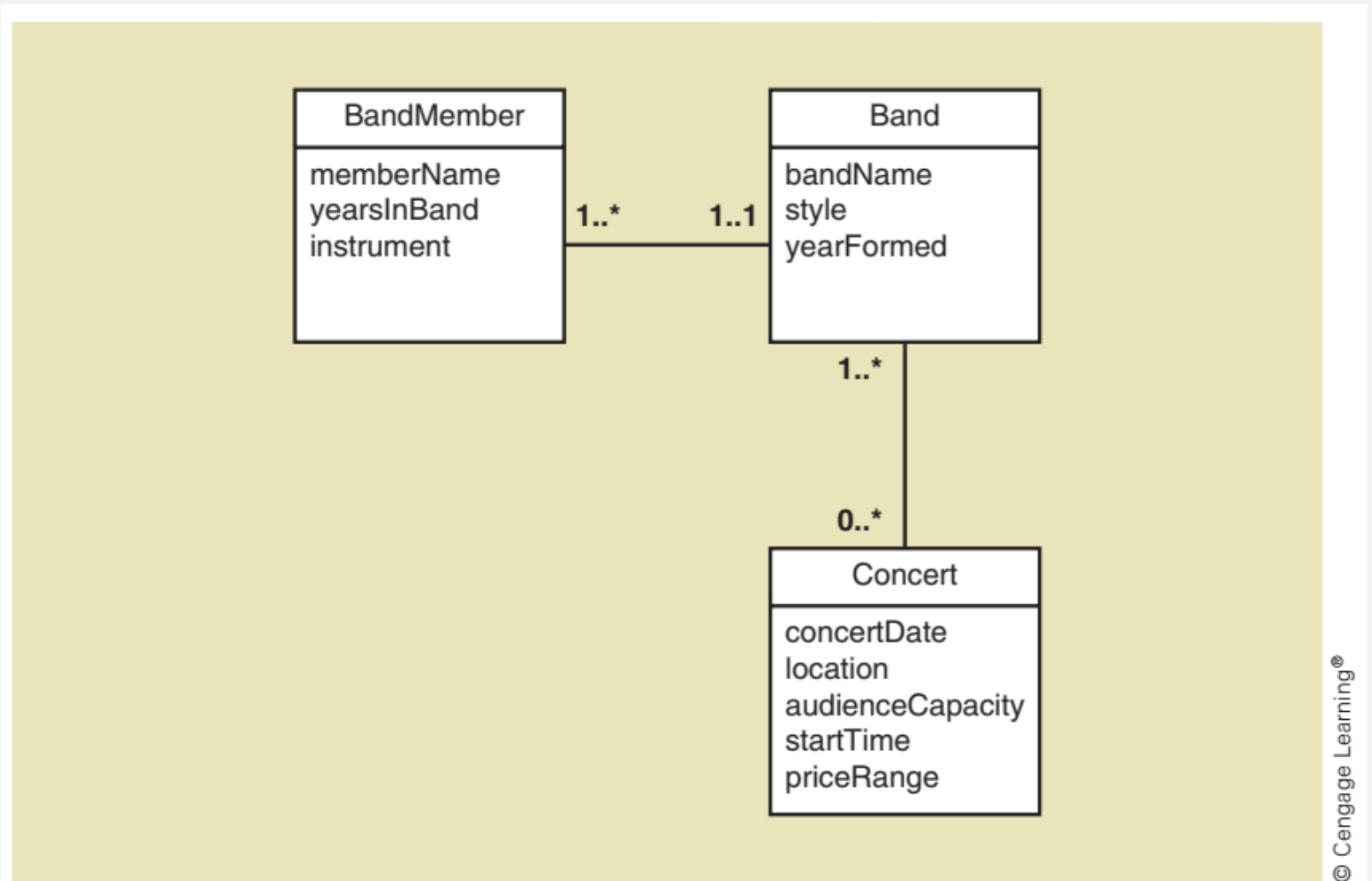
DOMAIN CLASS –EXAMPLE #3



DOMAIN CLASS –EXAMPLE #3

Class Activity #2

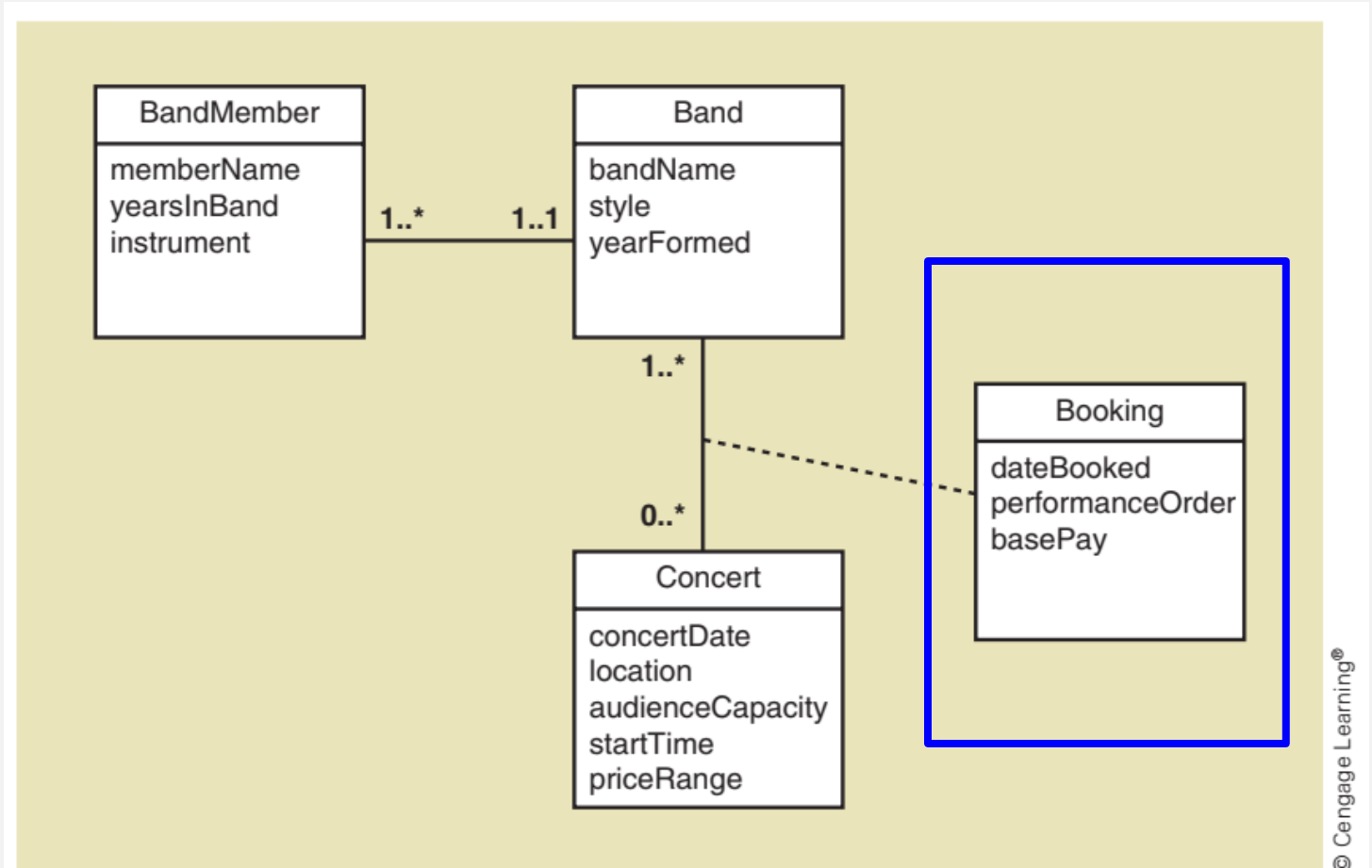
1. How many bands can a person play in?
2. For a band, how many concerts can it play in?
3. For a concert, how many bands may be playing?
4. What attributes can you use for keys?
5. Do you need to add “key” attributes?



DOMAIN CLASS –EXAMPLE #3: BAND WITH CONCERT BOOKING INFORMATION

Note: The association class (Booking) also provides a name and meaning for the association

1. Given the keys you identified, what is the key for the Booking class?
2. Does it uniquely identify instances?



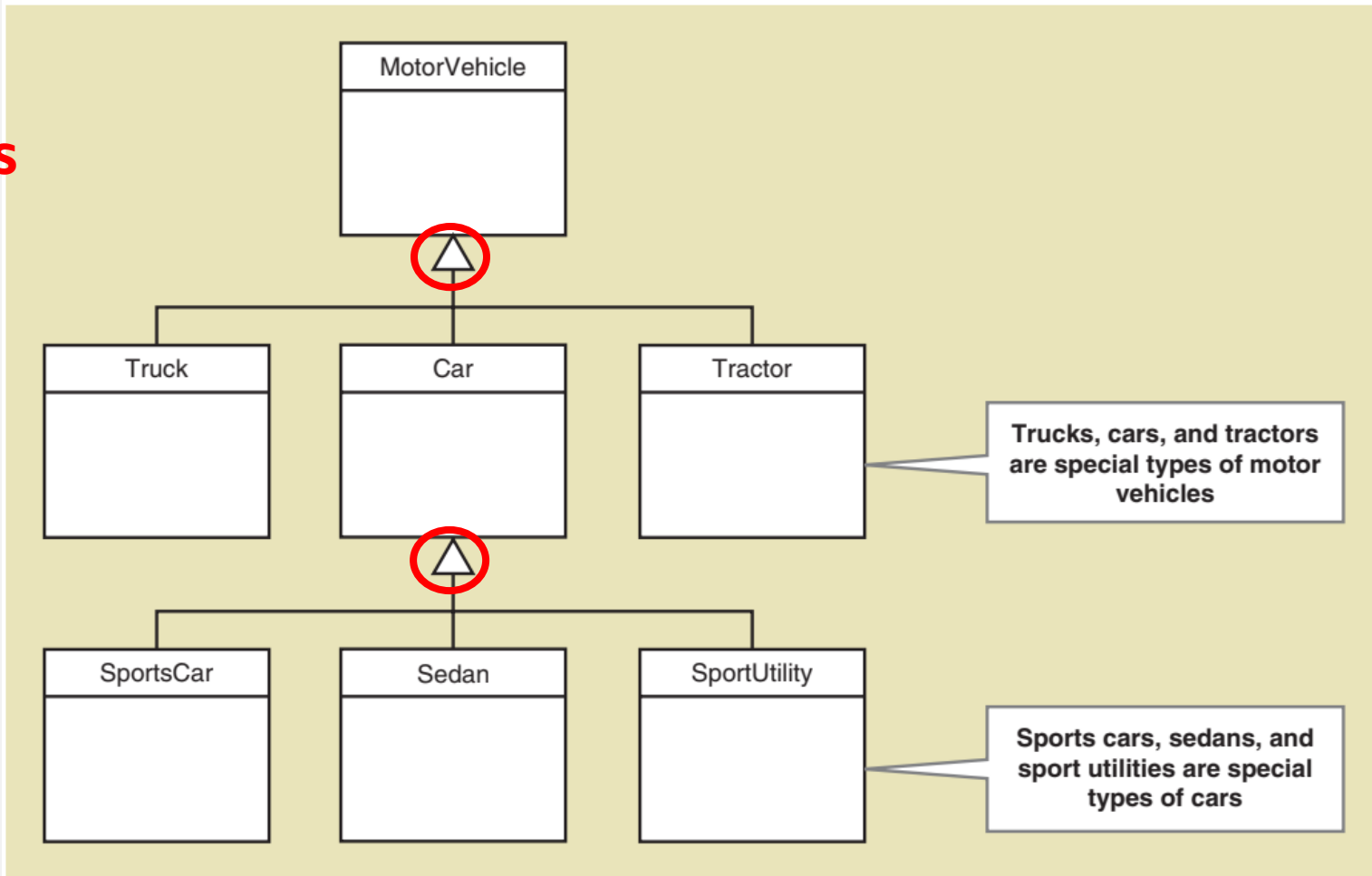
MORE COMPLEX ISSUES ABOUT CLASSES: GENERALIZATION/SPECIALIZATION RELATIONSHIPS

- Generalization/Specialization
 - ✓ A hierarchical relationship where subordinate classes are special types of the superior classes. Often called an Inheritance Hierarchy
- Superclass
 - ✓ the superior or more general class in a generalization/specialization hierarchy
- Subclass
 - ✓ the subordinate or more specialized class in a generalization/specialization hierarchy
- Inheritance
 - ✓ the concept that subclasses classes inherit characteristics of the more general superclass
 - ✓ allows subclasses to share characteristics of their superclass.

GENERALIZATION/SPECIALIZATION

SUPERCLASS

SUBCLASS



A hierarchical relationship where subordinate classes are special types of the superior classes.

Often called an Inheritance Hierarchy

GENERALIZATION/SPECIALIZATION: **INHERITANCE** FOR RMO

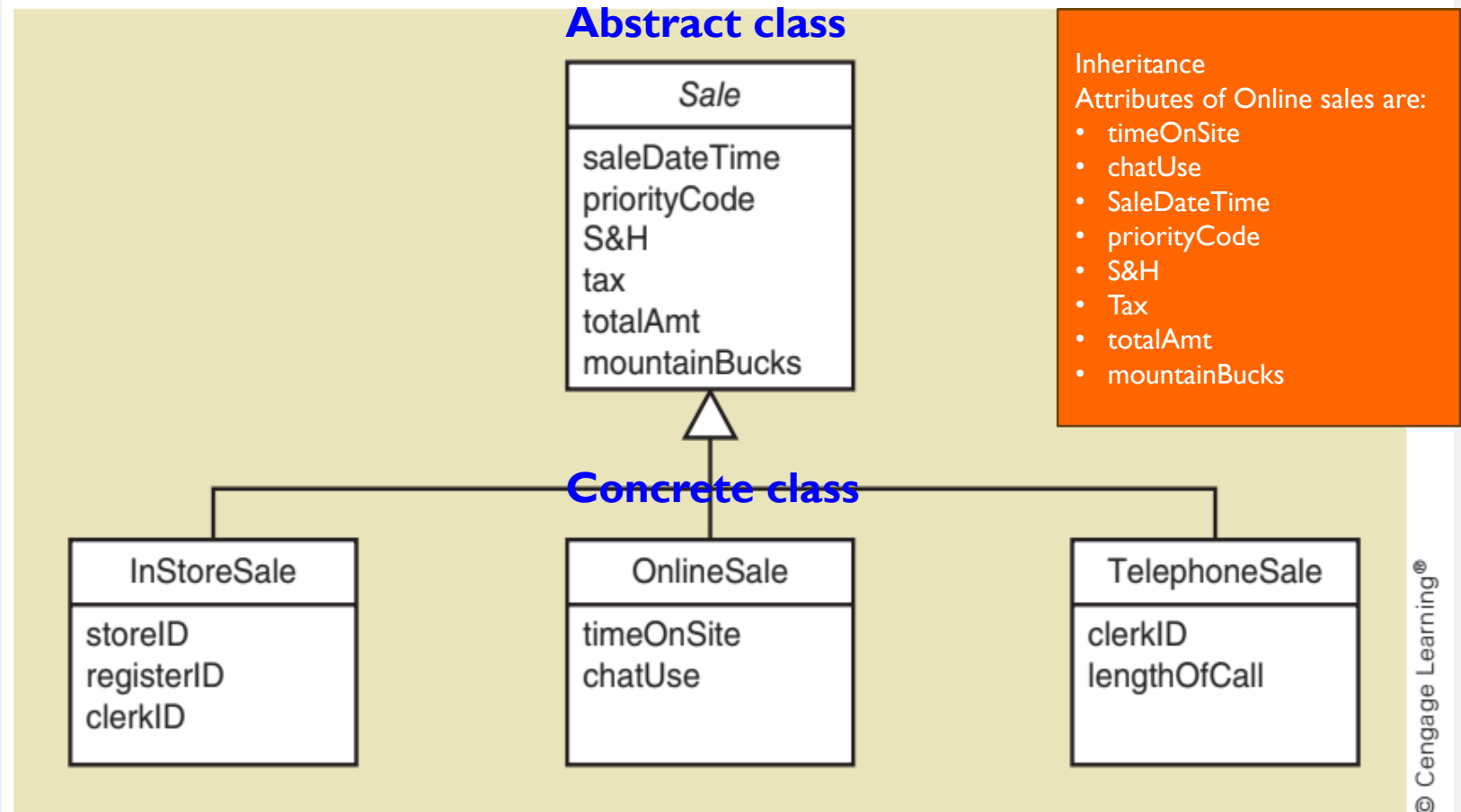
THREE TYPES OF SALES

Abstract class

- is a class that only exists so subclasses can inherit from it.
- In *Italics* (Sale)
- There is never an actual object simply called a Sale.
- Each sale must be one of the three subclasses

Concrete class

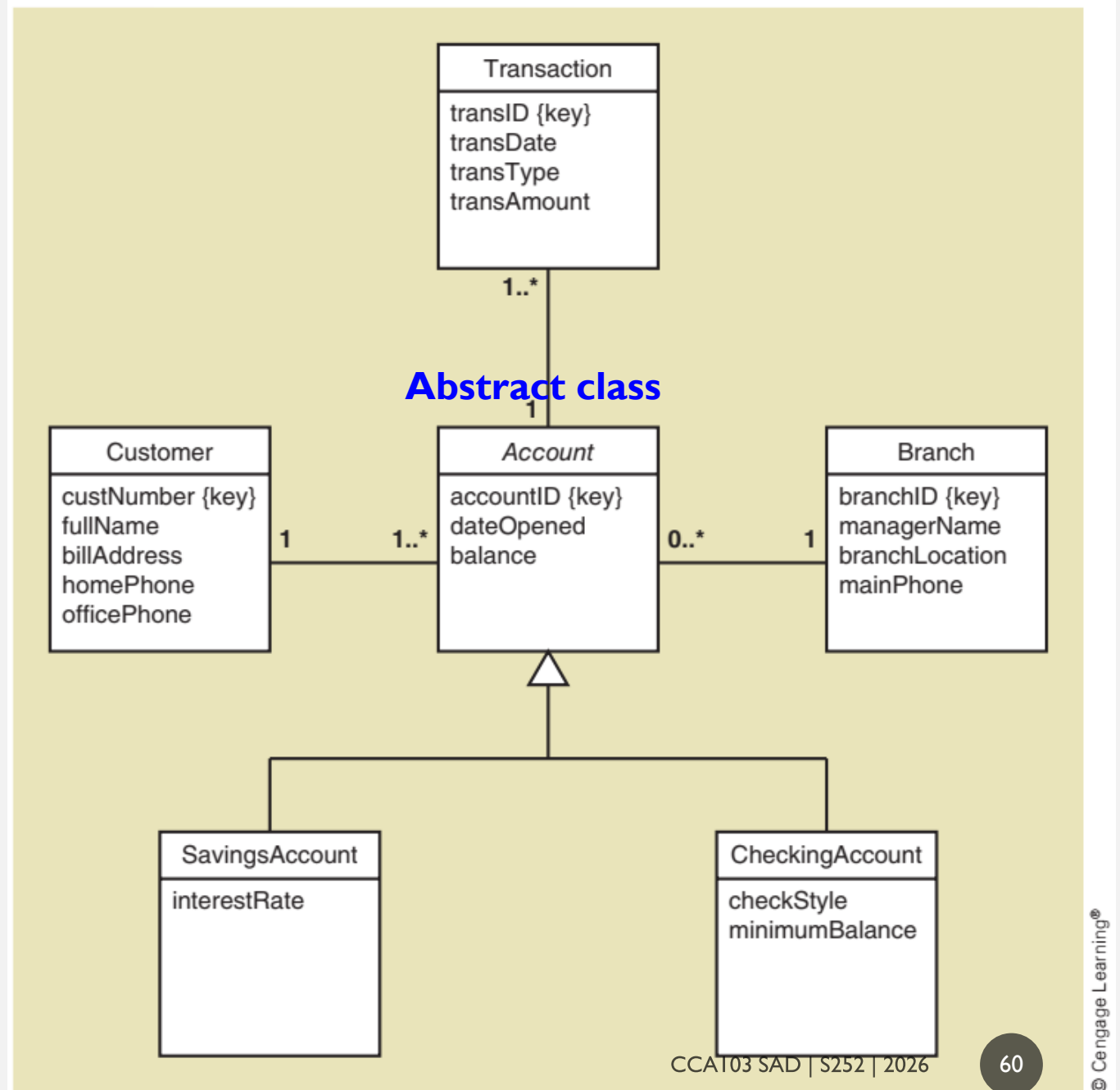
- is a class that does have actual objects.
- Sometimes, a superclass is abstract; sometimes, it is concrete, depending on the intention of the analyst



GENERALIZATION/SPECIALIZATION:
INHERITANCE FOR THE BANK WITH
SPECIAL TYPES OF ACCOUNTS

- A SavingsAccount has **4** attributes
- A CheckingAccount has **5** attributes

Note that each subclass also inherits an association with a Customer, optionally a Branch, and one or more Transactions.



MORE COMPLEX ISSUES ABOUT CLASSES: WHOLE PART RELATIONSHIPS

Whole-part relationships

- are used to show an association between one class and other classes that are parts of that class.
- There are two types of whole-part relationships: aggregation and composition

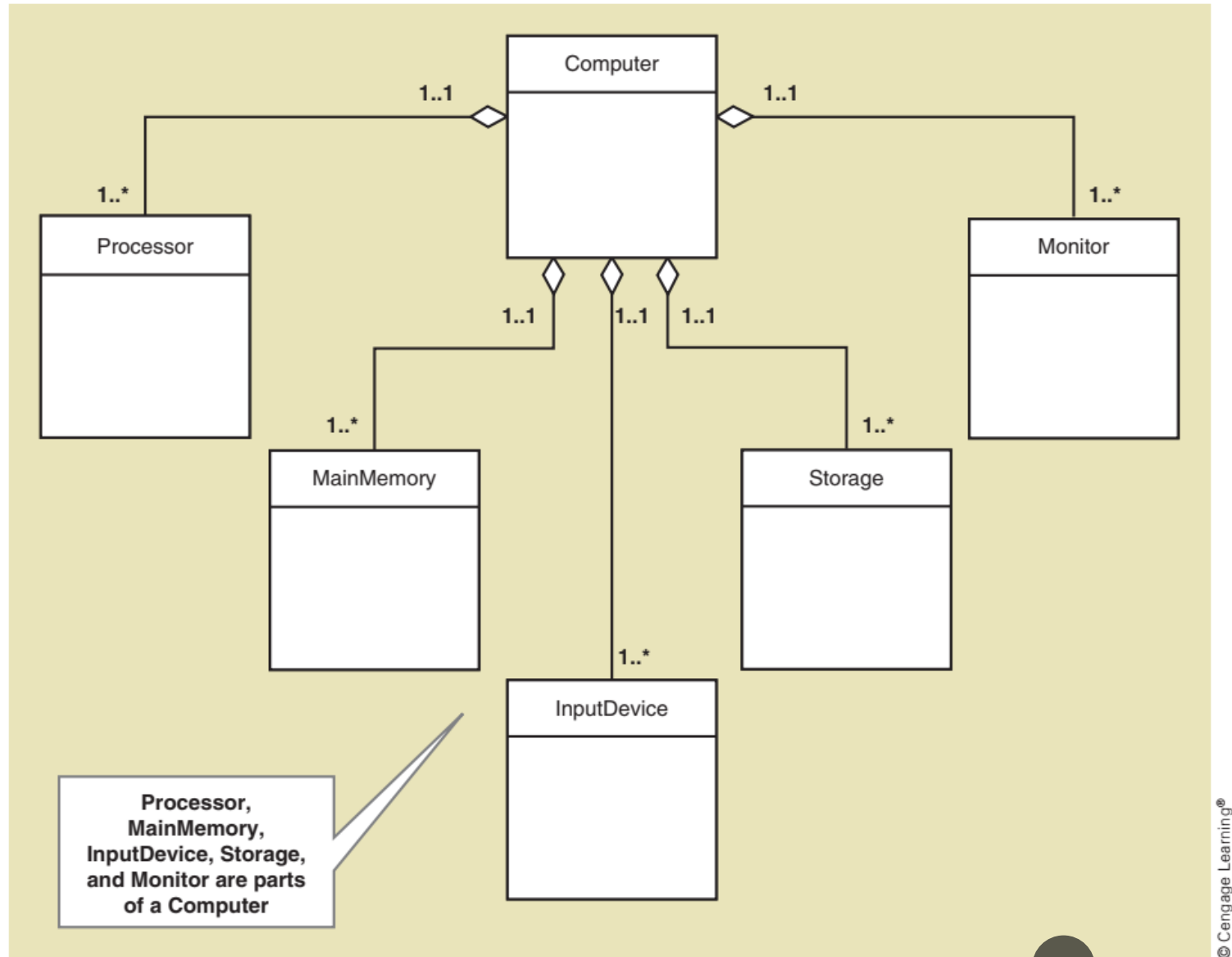
Aggregation

- Aggregation refers to a type of whole-part relationship between the aggregate (whole) and its components (parts), where the parts can exist separately.

Composition

- refers to whole-part relationships that are even stronger, where the parts, once associated, can no longer exist separately.

WHOLE PART RELATIONSHIPS: COMPUTER AND ITS PARTS



IN A NUTSHELL

There are three types of relationships in class diagrams

1. Association Relationships

- These are associations discussed previously, just like ERD relationships

2. Whole Part Relationships

- One class is a component or part of another class

3. Generalizations/Specialization Relationships

- Inheritance

Try not to confuse a relationship with an association

RMO Sales subsystem domain model class diagram

Association

Each customer can be associated with one or more sales.

Generalisation

There are three special types of sales shown in the inheritance relationships (inStore sale, onlineSale, and telephoneSale)

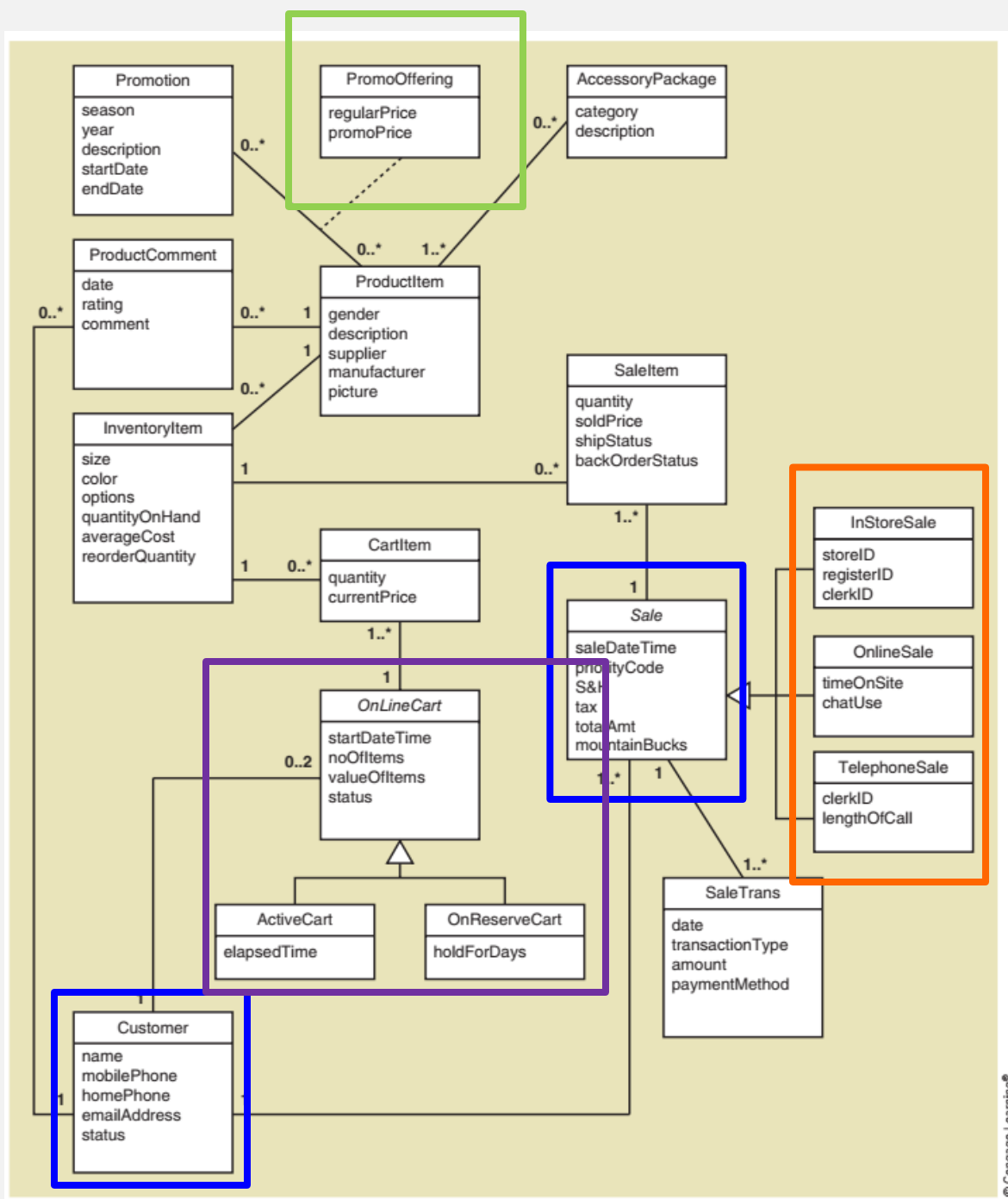
Inheritance

OnLineCart as Abstract Class

ActiveCart inherit attributes its abstract class with 5 attributes

Association class

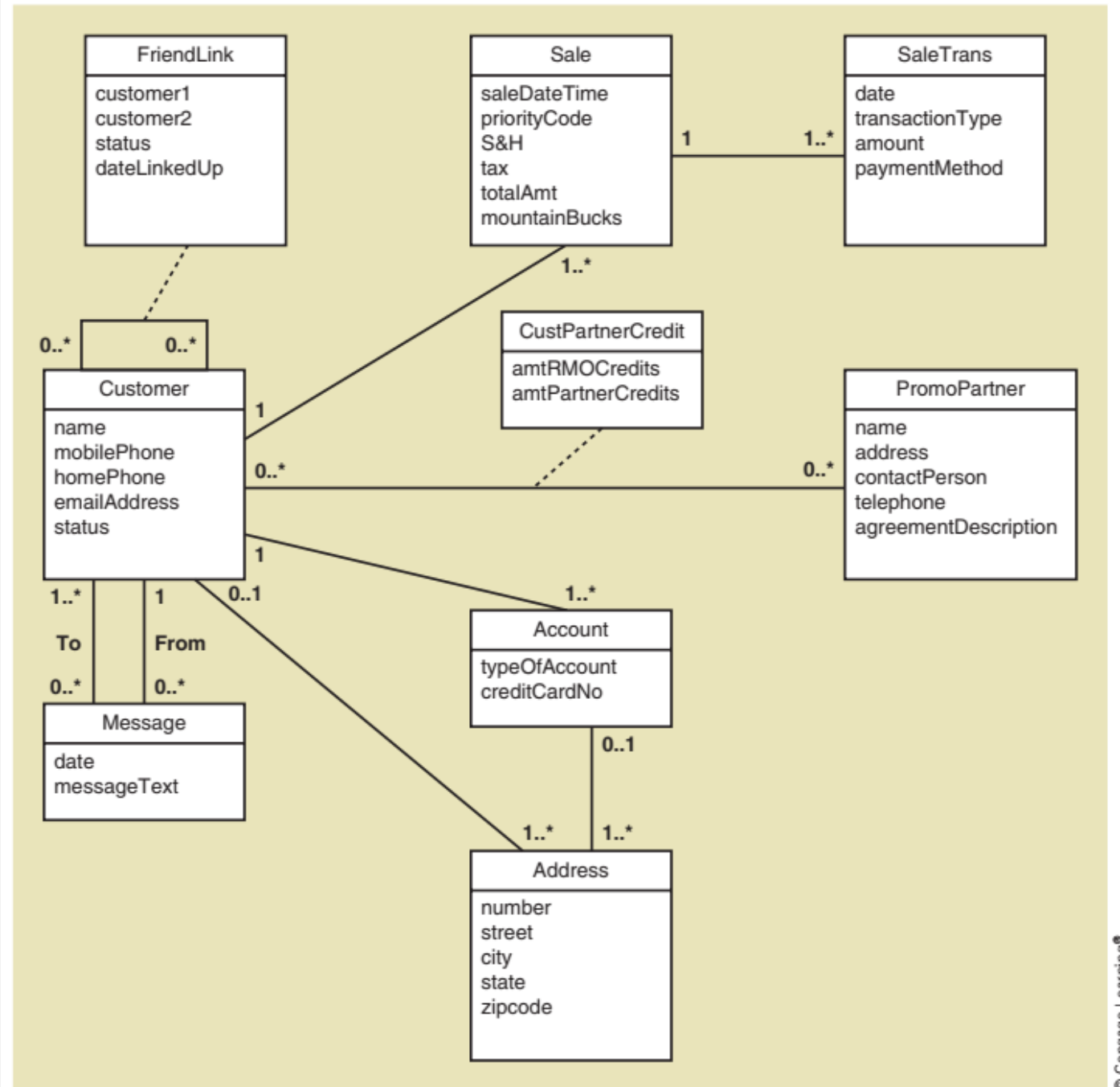
To handle and manage promotion offering information



Class Activity #3

Customer Account subsystem domain
in the model class diagram

1. Identify the following:
 - Association
 - Generalization
 - Association class
 - Inheritance
2. For every category, show the relationship/association and justify your answer.



OBJECT BEHAVIOR – STATE MACHINE DIAGRAM

- Each class has objects that may have status conditions or “states.”
- Object behavior consists of the various states and the movement between these states

State

- A state of an object is a condition that occurs during its life when it satisfies some criterion, performs some action, or waits for an event.
- For real-world objects, we mean the same thing whether we refer to the status condition of an object or talk about its state.

Transition

- A transition, then, is the movement of an object from one state to another state.
- Transitioning is the mechanism that causes an object to leave a state and change to a new state.

STATE MACHINE DIAGRAM – STATE & TRANSITION

STATE

- A state of an object is a condition that occurs during its life when it satisfies some criterion, performs some action, or waits for an event.
- The naming convention for status conditions helps identify valid states.
- A state might have a name of a simple condition, such as *On* or *In repair*.
- Other states are more active, with names consisting of gerunds or verb phrases, such as *Being shipped* or *Working*.
- The name of a state **should not be a noun** (which is an object); it should be something that describes the object.

TRANSITION

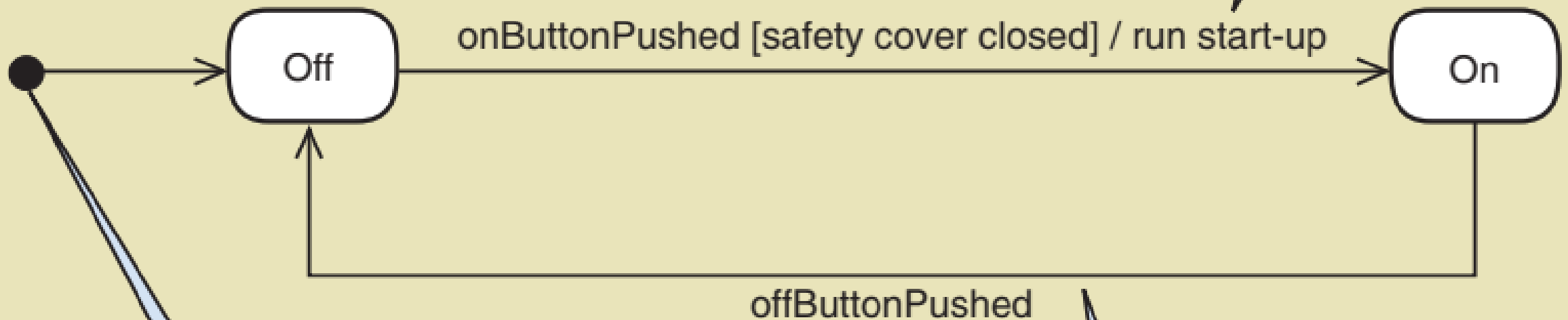
- A transition, then, is the movement of an object from one state to another state.
- Transitioning is the mechanism that causes an object to leave a state and change to a new state.
- Generally, transitions are short in duration—compared with states—and they cannot be interrupted.
- For example, you would say that a customer sale must be in a **Paid** for state before it can transition to a **Shipped** state.

STATE MACHINE DIAGRAM

1. State Machine Diagram - a diagram that shows the life of an object in states and transitions
2. Origin state - the original state of an object **before it begins a transition**
3. Destination state - the state to which an object **moves after completing a transition**
4. Pseudostate - the **starting point** in a state machine diagram. Noted by a black circle.
5. action-expression - some activity that must be completed as part of a transition
6. guard-condition - a true/false test to see whether a transition can fire

A State indicates a state of being of the object. Name it as a condition or a verb phrase.

Transition-name can have a trigger, a guard, and an action-expression.



The beginning pseudostate denotes the start of the state machine behavior for this object.

A Transition moves the object from the origin state to the destination state.

CONCURRENCY IN A STATE MACHINE DIAGRAM

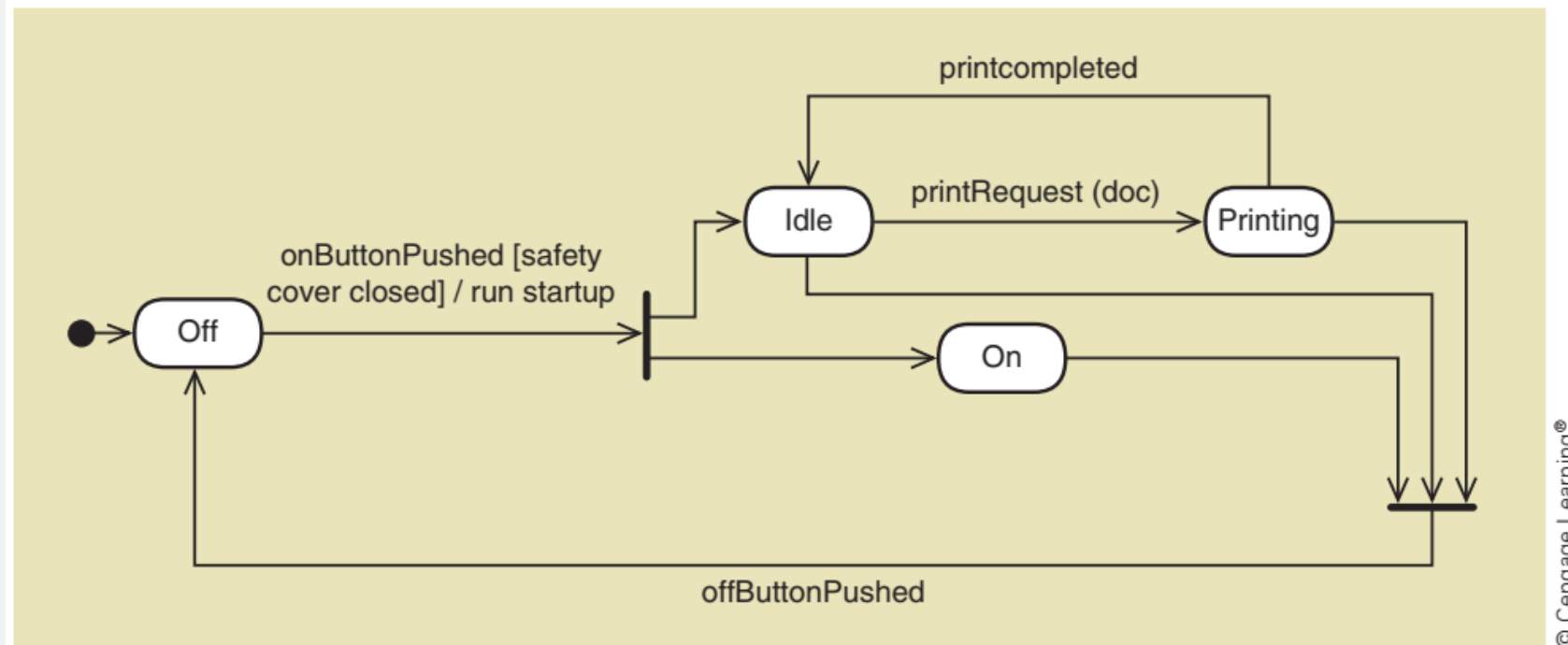
1. Concurrent states – when an object is in one or more states at the same time
2. Path – a sequential set of connected states and transitions
3. Concurrent paths – when multiple paths are being followed concurrently, i.e. when one or more states in one path are parallel to states in another path

PRINTER WITH CONCURRENT PATHS

- Sometimes an object will be in two states at the same time.
- For example, when the printer is in the **On** state, it can also be either Printing or **Idle**. There are several ways to express this concurrent condition
 1. Concurrent states – when an object is in one or more states at the same time
 2. Path – a sequential set of connected states and transitions
 3. Concurrent paths – when multiple paths are being followed concurrently, i.e. when one or more states in one path are parallel to states in another path

PRINTER WITH CONCURRENT PATHS

- Concurrent paths often shown by synchronization bars (same as Activity Diagram)
- Multiple exits from a state is an “OR” condition.
- Multiple exits from a synchronization bar is an “AND” condition.



CREATING A STATE MACHINE DIAGRAM

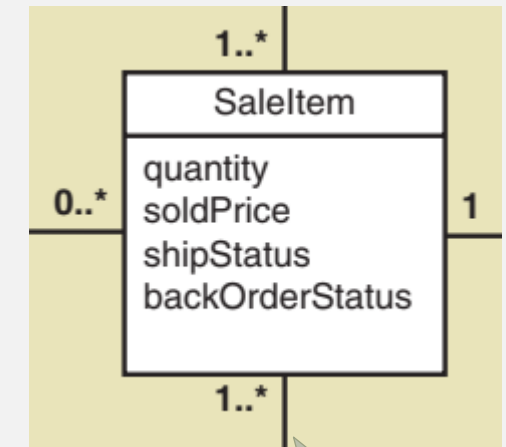
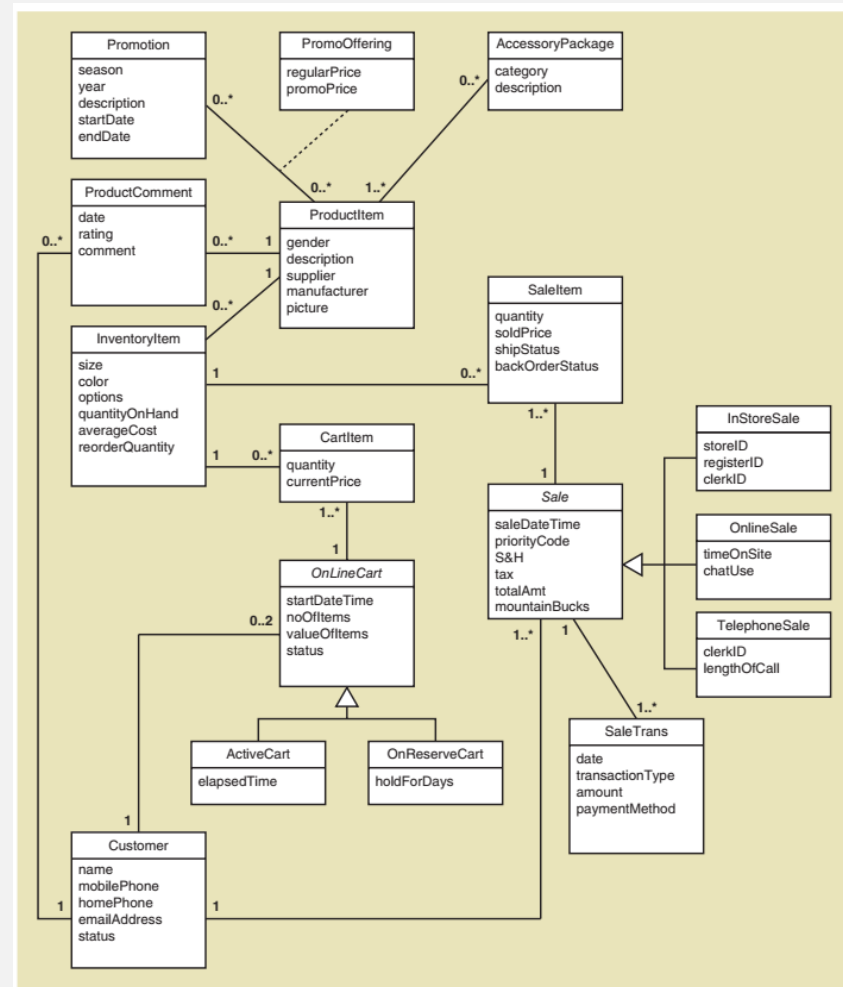
1. Review the class diagram and select classes that might require state machine diagrams
2. For each class, make a list of status conditions (states) you can identify
3. Begin building diagram fragments by identifying transitions that cause an object to leave the identified state
4. Sequence these states in the correct order and aggregate combinations into larger fragments
5. Review paths and look for independent, concurrent paths

CREATING A STATE MACHINE DIAGRAM

1. Look for additional transitions and test both directions
2. Expand each transition with appropriate message event, guard condition, and action expression
3. Review and test the state machine diagram for the class
 - ✓ Make sure states are really state for the object in the class
 - ✓ Follow the life cycle of an object coming into existence and being deleted
 - ✓ Be sure the diagram covers all exception conditions
 - ✓ Look again for concurrent paths and composite states

CREATING A STATE MACHINE DIAGRAM STEPS – SALEITEM #1

1. Choose SaleItem.
2. It has status conditions that need to be tracked



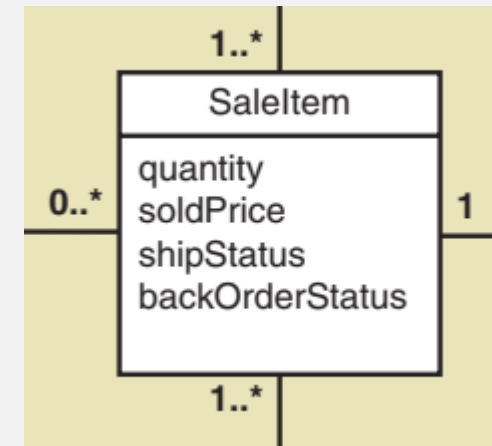
Possible state conditions

1. Ready to be shipped
2. On back order
3. Shipped

CREATING A STATE MACHINE DIAGRAM

STEPS – SALEITEM #2

1. Choose SaleItem.
2. It has status conditions that need to be tracked
3. List the states and exit transitions

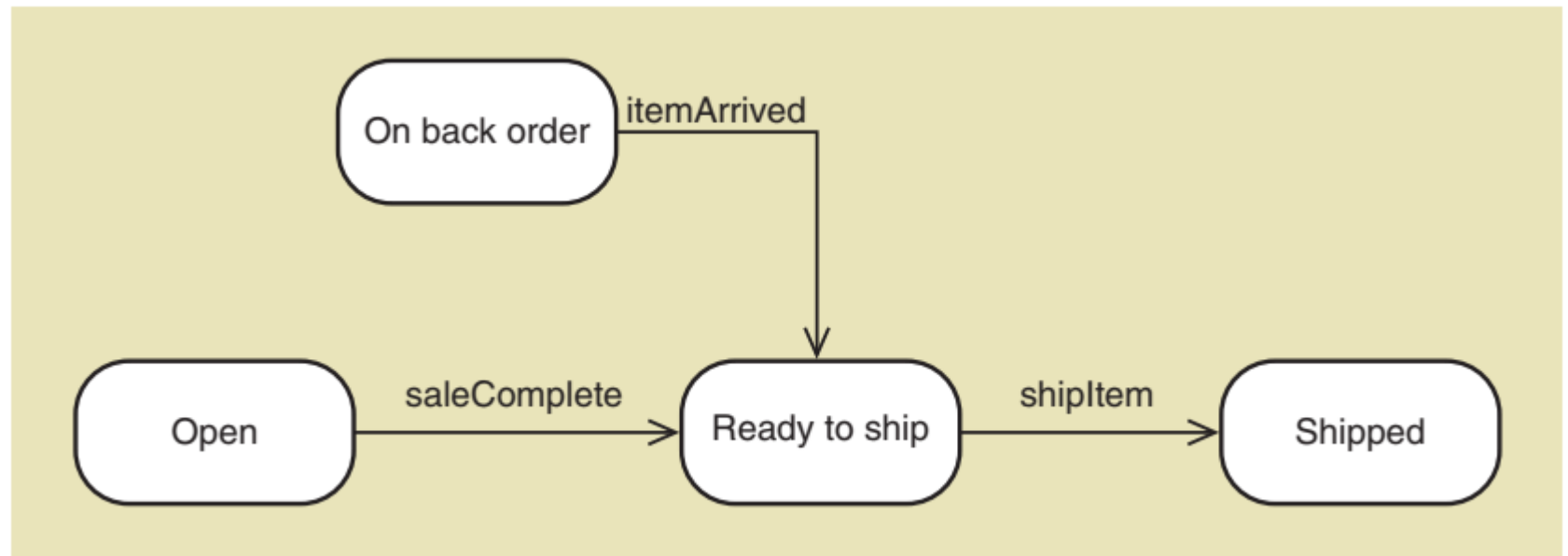


State	Transition causing exit
Open	saleComplete
Ready to Ship	shipItem
On back order	itemArrived
Shipped	No exit transition defined

CREATING A STATE MACHINE DIAGRAM

STEPS – SALEITEM #3

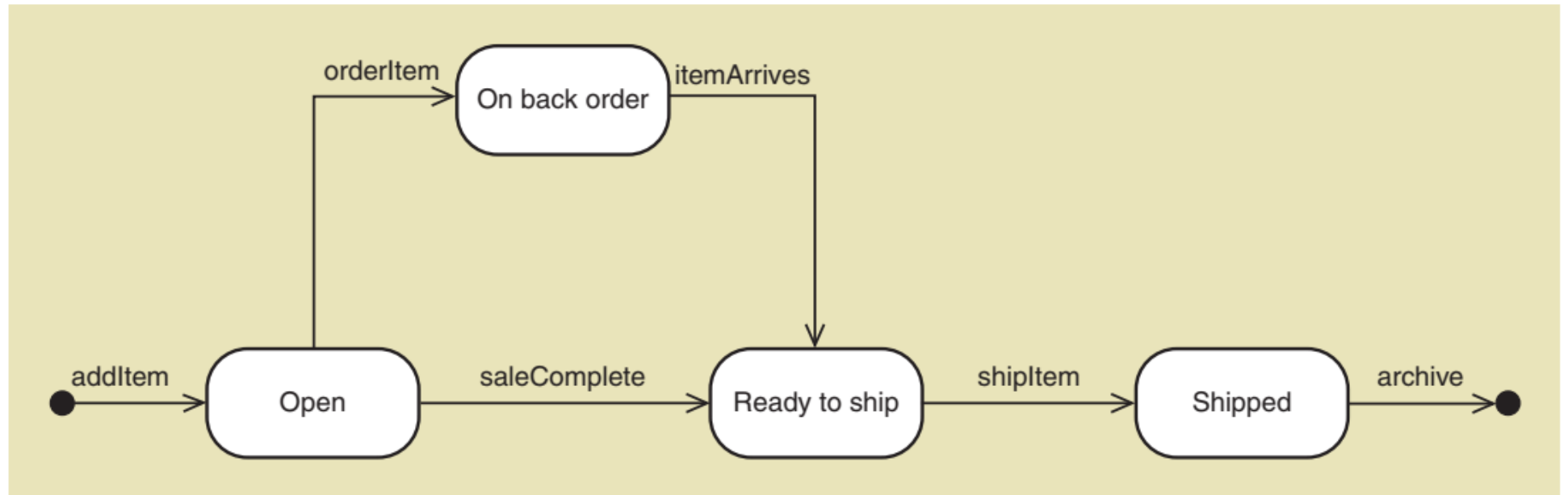
1. Build fragments
2. Sequence in correct order
3. Look for concurrent paths



CREATING A STATE MACHINE DIAGRAM

STEPS – SALEITEM #4

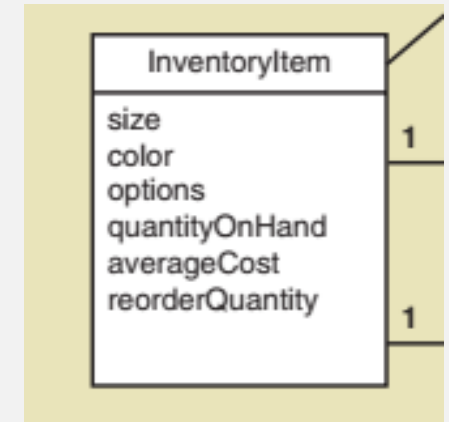
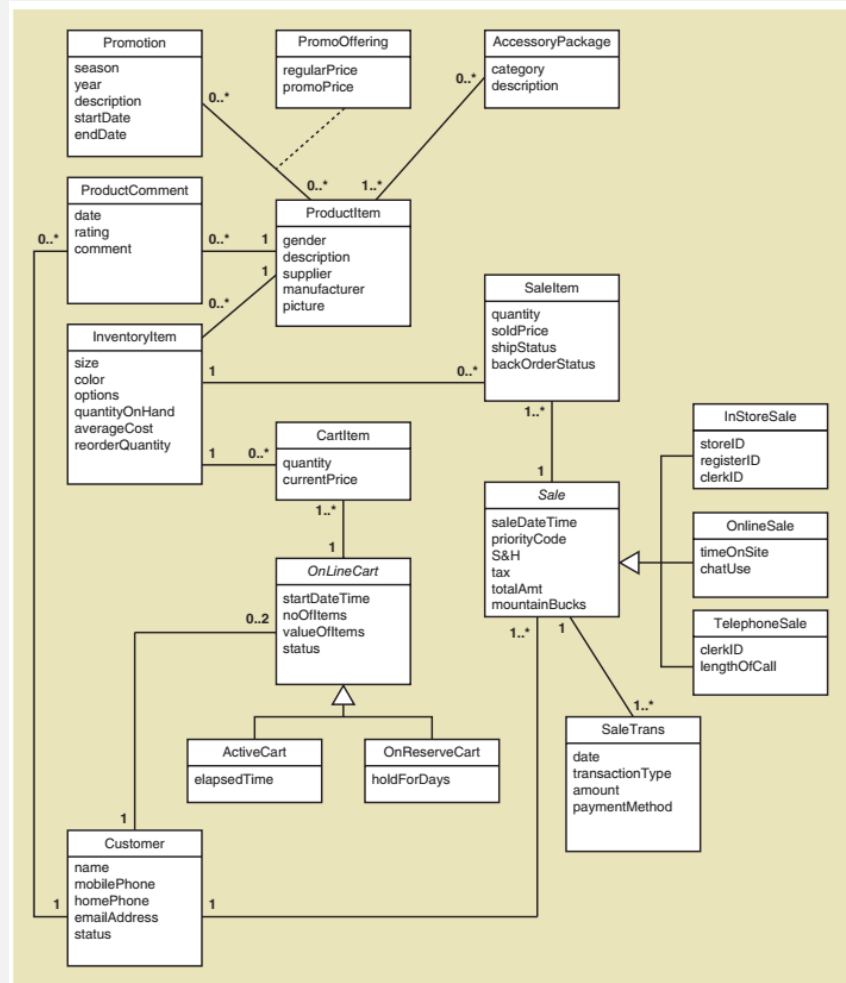
1. Add other required transitions
2. Expand with guard, action-expressions etc.
3. Review and test



CREATING A STATE MACHINE DIAGRAM

STEPS – INVENTORYITEM #1

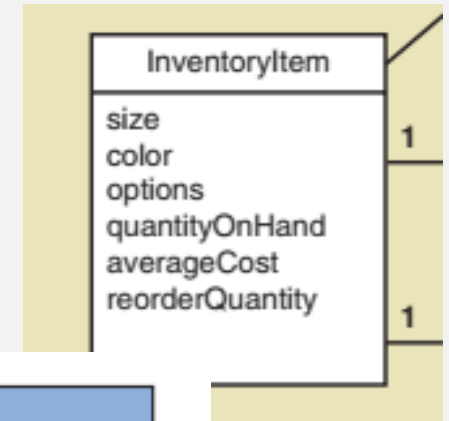
1. Choose InventoryItem.
2. It has status conditions that need to be tracked



CREATING A STATE MACHINE DIAGRAM

STEPS – INVENTORYITEM #2

1. List the states and exit transitions

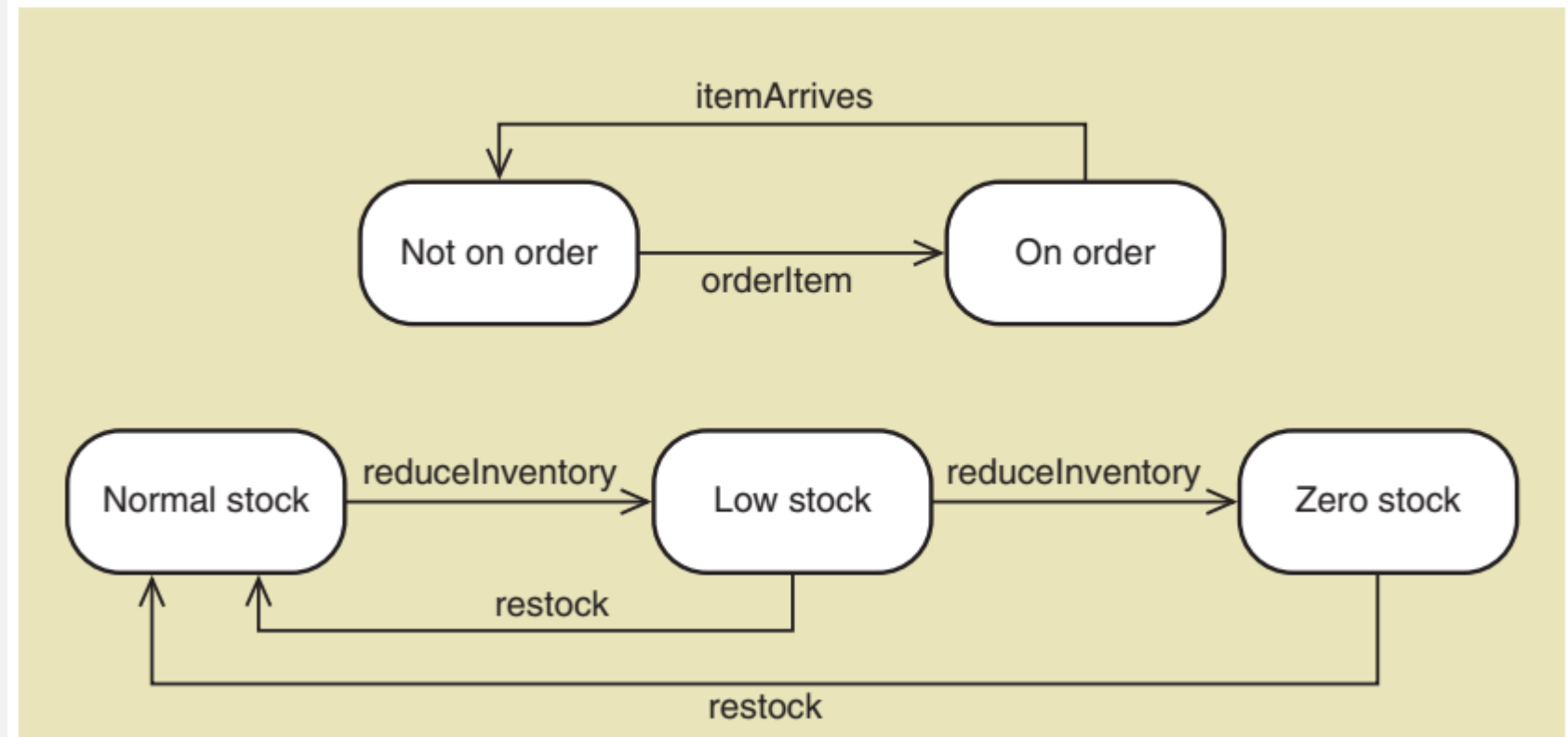


State	Transition causing exit
Normal stock	reduceInventory
Low stock	reduceInventory OR restock
Zero stock	removeItem OR restock
On order	itemArrives
Not on order	orderItem

CREATING A STATE MACHINE DIAGRAM

STEPS – INVENTORYITEM #3

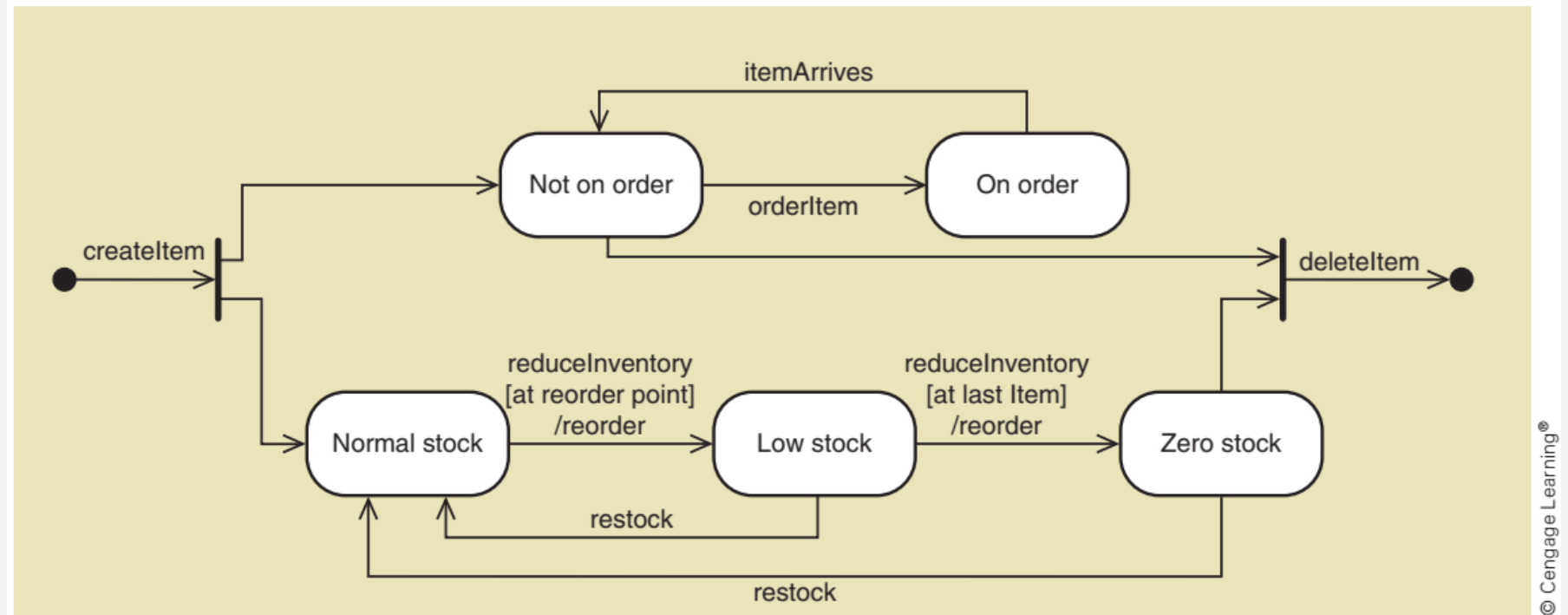
1. Build fragments
2. Sequence in correct order
3. Look for concurrent paths



CREATING A STATE MACHINE DIAGRAM

STEPS – INVENTORYITEM #4

1. Add other required transitions
2. Expand with guard, action-expressions etc.
3. Review and test





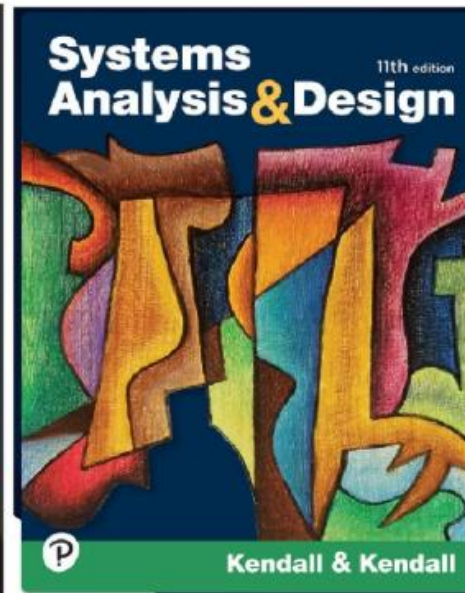
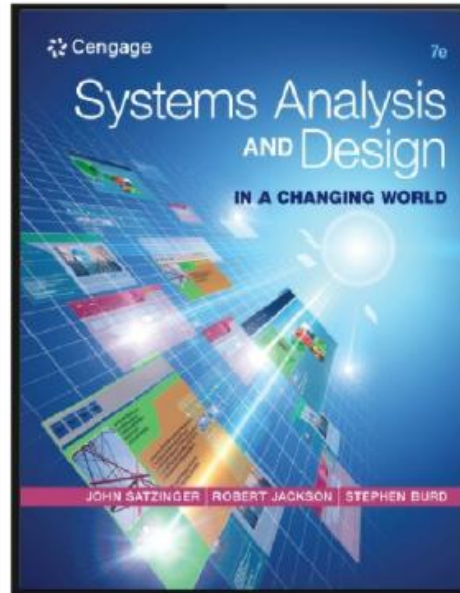
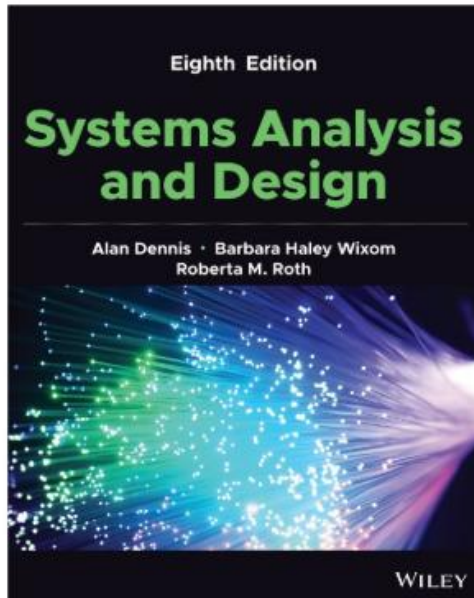
SUMMARY #1

1. “Things” in the problem domain are identified and modeled, called domain classes or data entities
2. Two techniques for identifying domain classes/data entities are the brainstorming technique and the noun technique
3. Domain classes have attributes and associations
4. Associations are naturally occurring relationships among classes, and associations have a minimum and a maximum multiplicity
5. Entity-relationship diagrams (ERDs) show the information about data entities
6. ERDs are often preferred by database analysts and are widely used
7. ERDs are not UML diagrams, and an association is called a relationship, multiplicity is called cardinality, and generalization/specialization (inheritance) and whole-part relationships are usually not shown

SUMMARY #2

1. The UML class diagram notation is used to create a domain model class diagram for a system. The domain model classes do not have methods because they are not yet software classes.
2. There are actually three UML class diagram relationships: association relationships, generalization/specialization (inheritance) relationships, and whole-part relationships
3. Other class diagram concepts are abstract versus concrete classes, compound attributes, composition and aggregation, association classes, superclasses, and subclasses
4. Some objects have a life cycle with status conditions that change and should be tracked
5. A State Machine Diagram tracks the behavior of these objects with states and transitions

REFERENCES



- (R1) Dennis, A. Wixom B.H., Roth R. M. (2021). Systems Analysis and Design (8th ed.). John Wiley & Sons
- (R2) Satzinger, J. W., Jackson, R. B., & Burd, S. D. (2015). Systems Analysis and Design in a Changing World (7th ed.). Course Technology CENGAGE Learning
- ~~(R3) Kendall, K. E., & Kendall, J. E. (2023). Systems Analysis and Design (11th ed.). Pearson~~